

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БАРАНОВИЧСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет Инженерный
Кафедра Информационных технологий и физико-математических
дисциплин

Дата регистрации работы в деканате _____
Дата регистрации работы на кафедре _____
Отметка о допуске к защите _____
Оценка за защиту _____

КУРСОВОЙ ПРОЕКТ

по дисциплине: «Программирование сетевых приложений»

Тема:

«Разработка клиент-серверного приложения для верификации цифровых
подписей с использованием машинного обучения»

Исполнитель:

_____ 3 курс, ИСТ-31 _____
_____ Олешкевич Кирилл Вадимович _____

Руководитель:

_____ Ст. Преподаватель _____
_____ Калько Алексей Игоревич _____

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БАРАНОВИЧСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

РЕЦЕНЗИЯ

на курсовой проект

(регистрационный №)

Студента Олешкевича Кирилла Вадимовича

Факультет Инженерный

Курс	3
------	---

Дисциплина Программирование сетевых приложений

Рецензент	Калько Алексей Игоревич
-----------	-------------------------

Дата получения к/п для рецензирования

Дата возвращения к/п после

рецензирования

Подпись преподавателя-рецензента

Текст рецензии:

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

РЕФЕРАТ

Курсовой проект: 46 с., 25 рис., 8 источников, 6 прил.

ВЕБ-ПРИЛОЖЕНИЕ, КЛИЕНТ-СЕРВЕР, ЦИФРОВЫЕ ПОДПИСИ, МАШИННОЕ ОБУЧЕНИЕ, TYPESCRIPT, PYTHON, POSTGRESQL, NEXT.JS, FASTAPI, GOOGLE COLAB.

Объектом исследования является клиент-серверное веб-приложение для верификации цифровых подписей с использованием машинного обучения.

Цель курсового проекта заключается в создании веб-приложения, позволяющего пользователям создавать цифровые подписи, хранить их в базе данных и автоматически проверять их подлинность с помощью обученной нейронной сети. Основной задачей проекта является упрощение процесса верификации подписей и повышение уровня безопасности при работе с цифровыми документами.

В ходе работы были применены следующие технологии и инструменты: Next.js и React для разработки Frontend приложения, FastAPI для создания Backend inference сервера, PyTorch для работы с машинным обучением, Supabase (PostgreSQL) для управления базами данных, Vercel Blob Storage для хранения ML моделей, Render для развертывания Backend сервера, Jest и Playwright для тестирования приложения.

В процессе работы проведены следующие исследования и разработки: проанализированы существующие подходы к верификации подписей, разработана архитектура распределенного приложения, реализован механизм сбора подписей через Canvas интерфейс, создана система предобработки данных и управления ML моделями, реализована система аутентификации с различными ролями пользователей, а также проведено комплексное тестирование приложения.

Практическое применение проекта предполагает его внедрение в банковские учреждения, юридические компании, системы электронного

документооборота и образовательные учреждения для автоматизации процесса проверки подлинности подписей на документах.

Технико-экономическая актуальность проекта объясняется необходимостью создания эффективных инструментов для автоматизации верификации подписей в условиях цифровизации документооборота. Система позволяет оптимизировать трудозатраты на проверку подлинности подписей, сократить время обработки документов и повысить точность определения подделок за счет применения машинного обучения.

Социальная значимость проекта заключается в его способности улучшить процесс работы с цифровыми документами, способствуя более эффективному и безопасному взаимодействию между организациями и их клиентами, а также развитию цифровой экономики.

Автор подтверждает, что приведённый в работе расчётно-аналитический материал правильно и объективно отражает состояние исследуемого процесса, а все заимствованные из литературных и других источников теоретические, методологические и методические положения и концепции сопровождаются ссылками на их авторов.

(подпись обучающегося)

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	5
1.1 Постановка задачи.....	5
1.2 Описание архитектуры приложения.....	8
1.3 Описание использованных в разработке компонентов.....	19
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	25
2.1 Руководство пользователя.....	25
2.2 Результаты тестирования программного продукта.....	39
ЗАКЛЮЧЕНИЕ.....	43
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	46
ПРИЛОЖЕНИЯ.....	47

ВВЕДЕНИЕ

В современном мире цифровые подписи становятся все более важным инструментом для аутентификации и верификации документов. С развитием технологий машинного обучения появилась возможность автоматизировать процесс проверки подлинности подписей, что значительно повышает безопасность и эффективность работы с документами.

Актуальность темы курсового проекта обусловлена необходимостью создания надежной системы для верификации цифровых подписей с использованием современных сетевых технологий и методов машинного обучения. Такие системы находят применение в банковской сфере, юридических документах, системах электронного документооборота и других областях, где требуется подтверждение подлинности подписи.

Целью курсового проекта является разработка клиент-серверного приложения для верификации цифровых подписей с использованием машинного обучения. Приложение должно обеспечивать сбор подписей, их хранение в базе данных и автоматическую верификацию с помощью обученной нейронной сети.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Разработка Frontend приложения на базе Next.js для сбора и отображения цифровых подписей.
2. Разработка Backend inference сервера на базе FastAPI для верификации подписей с использованием обученной ML модели.
3. Интеграция с базой данных Supabase для хранения подписей и метаданных.
4. Реализация системы аутентификации и авторизации пользователей с различными ролями.
5. Разработка механизма предобработки данных подписей для подачи в нейронную сеть.

6. Реализация API для взаимодействия между клиентской и серверной частями приложения.

7. Составить подробное руководство пользователя, документирующее порядок работы с системой для каждой категории пользователей.

Курсовой проект состоит из двух основных частей. В теоретической части рассматривается постановка задачи, описывается архитектура приложения и используемые компоненты. В практической части представлено руководство пользователя и результаты тестирования разработанного программного продукта.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Постановка задачи

Заданием курсового проекта является разработка клиент-серверного приложения для верификации цифровых подписей с использованием машинного обучения. Система должна обеспечивать сбор, хранение и автоматическую проверку подлинности цифровых подписей путем сравнения их с эталонными образцами.

Разработать распределенное веб-приложение, состоящее из следующих компонентов:

1. Frontend приложение (Next.js) — веб-интерфейс для работы пользователей.
2. Backend inference сервер (FastAPI) — сервер для верификации подписей с использованием ML модели.
3. База данных (Supabase/PostgreSQL) — хранилище подписей и метаданных.

Требования к функциональности системы:

- Регистрация и аутентификация пользователей с различными ролями (user, mod, admin).
- Создание и сохранение цифровых подписей через Canvas интерфейс.
- Хранение подписей в формате CSV (точки с координатами x, y, давлением p и временем t).
- Верификация подписей путем сравнения с оригиналом с использованием ML модели.
- Управление подписями (просмотр, удаление, модерация).
- Управление ML моделями (загрузка, активация, переключение версий).
- Статистика и аналитика по подписям

Технические требования:

- Frontend: Next.js 16, React 19, TypeScript.
- Backend: FastAPI, PyTorch для ML inference.
- Database: PostgreSQL.
- Протоколы: HTTP/HTTPS, REST API.
- Deployment: Vercel (Frontend), Render (Backend), Supabase (Database).

Пользовательские роли:

- Guest — гость-не авторизованный пользователь, может попробовать подделать чужую подпись.
- User — обычный пользователь, может создавать и верифицировать свои подписи, подделывать чужие подписи.
- Mod — модератор, имеет права пользователя плюс модерация подписей и управление датасетом.
- Admin — администратор, имеет все права плюс управление моделями и пользователями.

Use-case диаграмма приведена на рисунке 1.1.

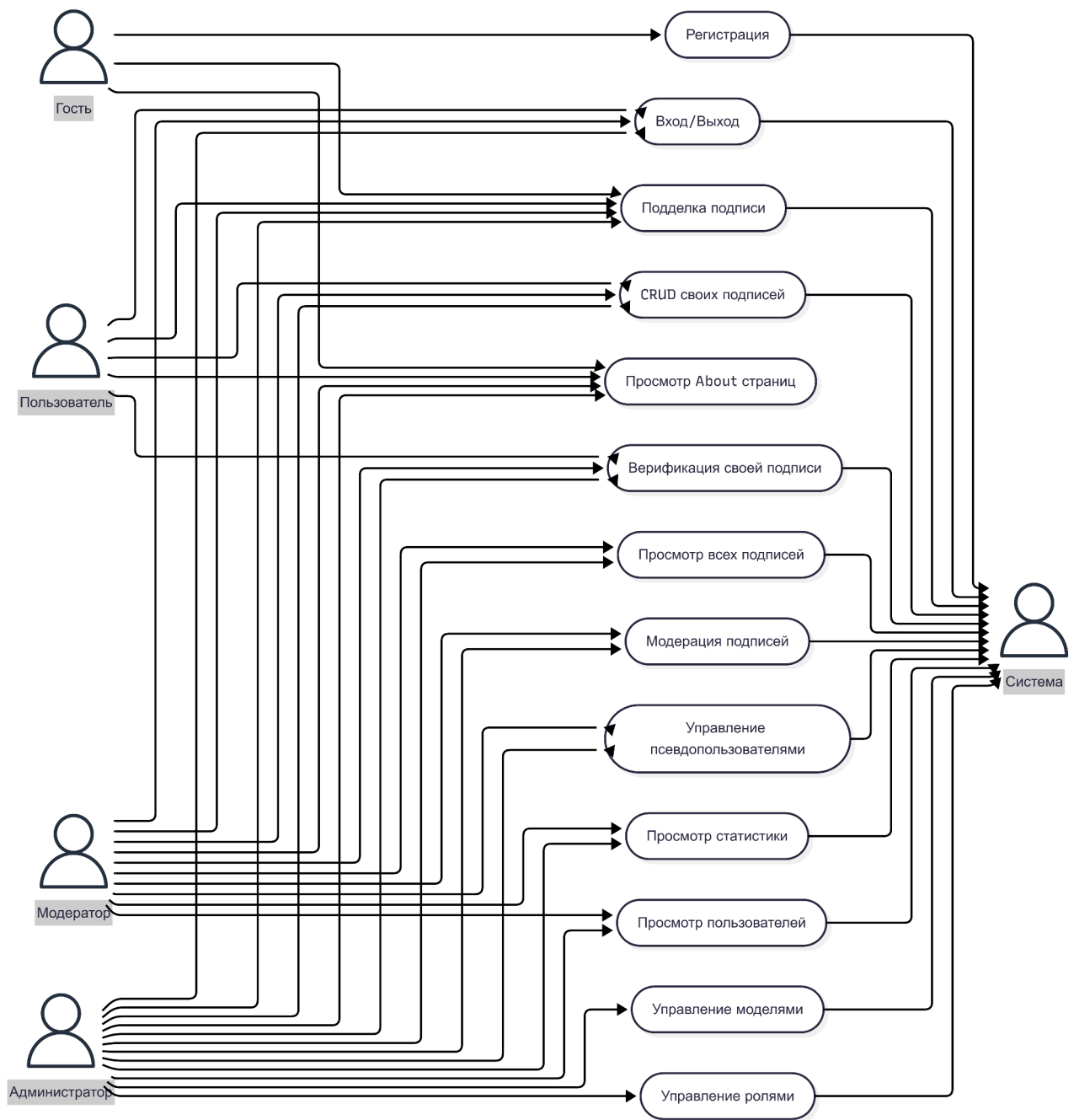


Рисунок 1.1 — Use-case диаграмма администратора

Входные данные:

- Подпись в формате массива точек: { timestamp, x, y, pressure }[].
- Тип устройства ввода: mouse | touch | pen.
- Идентификатор оригинальной подписи для верификации (UUID).
- Идентификатор пользователя (UUID), если авторизованн.

Выходные данные:

- Результат верификации: `{ similarity_score: number, is_forgery: boolean }`.
- Идентификатор сохраненной подписи (UUID).
- Метаданные подписи: дата создания, тип ввода, статистика.

Разработанная система обеспечивает:

1. Верификацию подписей — автоматическое определение подлинности подписи путем сравнения с оригиналом с использованием обученной нейронной сети. Система вычисляет косинусное сходство между эмбедами подписей и сравнивает его с пороговым значением.
2. Управление подписями — создание, просмотр, удаление и модерация цифровых подписей. Пользователи могут сохранять свои подписи, просматривать историю, модераторы могут управлять подписями всех пользователей для формирования датасета.
3. Управление моделями — загрузка новых версий ML моделей, их активация и переключение без остановки сервера (hotswap). Администраторы могут загружать обученные модели в формате PyTorch (.pt файлы) вместе с определениями архитектуры (.py файлы).
4. Система аутентификации — безопасная регистрация и вход пользователей через Supabase Auth с поддержкой JWT токенов и Row Level Security (RLS) политик в базе данных.
5. API для интеграции — RESTful API endpoints для взаимодействия между frontend и backend компонентами, а также для внешней интеграции.

1.2 Описание архитектуры приложения

Разработанное приложение представляет собой распределенную клиент-серверную систему, построенную на основе архитектуры с разделением на три основных компонента: Frontend приложение, Backend inference сервер и база

данных. Компоненты взаимодействуют через REST API и используют современные сетевые протоколы HTTP/HTTPS.

Система построена по принципу микросервисной архитектуры, где каждый компонент выполняет свою специфическую функцию:

- Frontend (Next.js) — предоставляет пользовательский интерфейс и обрабатывает взаимодействие с пользователем.
- Backend Inference (FastAPI) — выполняет верификацию подписей с использованием ML модели.
- Database (Supabase/PostgreSQL) — хранит данные о пользователях, подписях и метаданные.
- External Services — Vercel Blob Storage для хранения ML моделей, Google Colab для обучения моделей.

Взаимодействие между компонентами происходит через REST API endpoints. Frontend отправляет HTTP запросы к Next.js API Routes, которые в свою очередь взаимодействуют с Supabase для работы с базой данных и с FastAPI сервером для верификации подписей.

Frontend приложение организовано по структуре Next.js App Router и состоит из следующих модулей:

- UI Components (components/) — React компоненты для интерфейса пользователя, включая компоненты для работы с подписями, формами аутентификации, навигацией.
- API Routes (app/api/) — серверные endpoints Next.js для обработки запросов от клиента, валидации данных и взаимодействия с базой данных.
- Pages (app/) — страницы приложения, включая главную страницу, страницы создания и просмотра подписей, административные панели.
- Lib (lib/) — утилиты, клиенты для работы с Supabase и Inference API, типы TypeScript, хуки React.

Основные классы и компоненты Frontend:

- SignatureCanvas — компонент для рисования подписи на Canvas.

- InferenceClient — клиент для взаимодействия с Inference API
- SupabaseClient — клиент для работы с Supabase

Backend inference сервер построен на FastAPI и состоит из следующих модулей:

- Routes (routes/) — API endpoints для верификации подписей, управления моделями, health checks.
- Utils (utils/) — утилиты для загрузки моделей, предобработки данных, работы с Supabase и Blob Storage.
- Models (models/) — определения архитектур PyTorch моделей.
- Dependencies (dependencies.py) — FastAPI dependencies для инъекции зависимостей.

Основные классы Backend:

- ModelManager — управляет загрузкой, переключением и хранением ML моделей.
- ModelLoader — загружает PyTorch модели и выполняет inference.
- Preprocessing — предобрабатывает данные подписей для подачи в модель, позволяет поддерживать переменное количество входных признаков.
- BlobClient — клиент для работы с Vercel Blob Storage.
- SupabaseClient — клиент для работы с Supabase.

База данных состоит из следующих основных таблиц:

- profiles — профили пользователей с ролями (user, mod, admin), имеет связь 1:1 с системной таблицей auth.users.
- genuine_signatures — подлинные подписи.
- forged_signatures — поддельные подписи.
- pseudousers — псевдопользователи для внешних данных.

База данных использует Row Level Security (RLS) для обеспечения безопасности данных на уровне строк. Каждый пользователь может видеть только свои данные, модераторы имеют доступ к данным всех пользователей, администраторы имеют полный доступ.

Взаимодействие между компонентами происходит по следующей схеме:

1. Пользователь взаимодействует с Frontend через веб-браузер.
2. Frontend отправляет HTTP запросы к Next.js API Routes.
3. API Routes проверяют аутентификацию через Supabase Auth.
4. API Routes взаимодействуют с Supabase Database для чтения/записи данных.
5. Для верификации подписей Frontend отправляет запросы к FastAPI Inference серверу.
6. Inference сервер получает данные подписей из Supabase, предобрабатывает их и использует ML модель для генерации эмбеддингов.
7. Результаты верификации возвращаются обратно в Frontend.

Поток создания подписи:

1. Пользователь рисует подпись на Canvas компоненте.
2. Frontend собирает точки подписи (x, y, pressure, timestamp).
3. При нажатии "Сохранить" Frontend отправляет POST запрос к `/api/signatures`.
4. Next.js API Route проверяет JWT токен через Supabase Auth.
5. Данные валидируются и преобразуются в CSV формат.
6. Подпись сохраняется в таблицу `genuine_signatures` через Supabase.
7. Возвращается ID сохраненной подписи.

Поток верификации подписи:

1. Пользователь выбирает оригинальную подпись и рисует новую подпись для проверки.
2. Frontend отправляет POST запрос к `/forgery-by-data` на Inference сервер.
3. Inference сервер получает оригинальную подпись из Supabase по ID.
4. Обе подписи предобрабатываются (нормализация, вычисление производных признаков).
5. Предобработанные данные подаются в ML модель для генерации эмбеддингов.

6. Вычисляется косинусное сходство между эмбедингами.
7. Результат сравнивается с пороговым значением.
8. Возвращается результат верификации: { `similarity_score`, `is_forgery`, `threshold` }.

Поток обучения модели:

1. Администратор запускает обучение в Google Colab.
2. Training notebook запрашивает подписи из Supabase с флагом `mod_for_dataset = true` с авторизацией через Supabase Auth.
3. Создается LMDB датасет из полученных подписей.
4. Инициализируется PyTorch модель (SignatureEncoder).
5. Выполняется обучение модели с использованием Triplet Loss или Contrastive Loss.
6. После обучения модель сохраняется в формате .pt.
7. Администратор загружает модель через Frontend API.
8. ModelManager на Inference сервере загружает и активирует новую модель.

Диаграммы потоков данных создания и верификации подписи приведены в приложении А.

Основные классы системы и их взаимосвязи представлены на UML диаграммах классов. Диаграммы включают ключевые классы Frontend и Backend, которые выполняют взаимодействия и обработку данных.

Общая диаграмма (рисунок 1.2) показывает основные компоненты системы и их взаимодействие на высоком уровне.

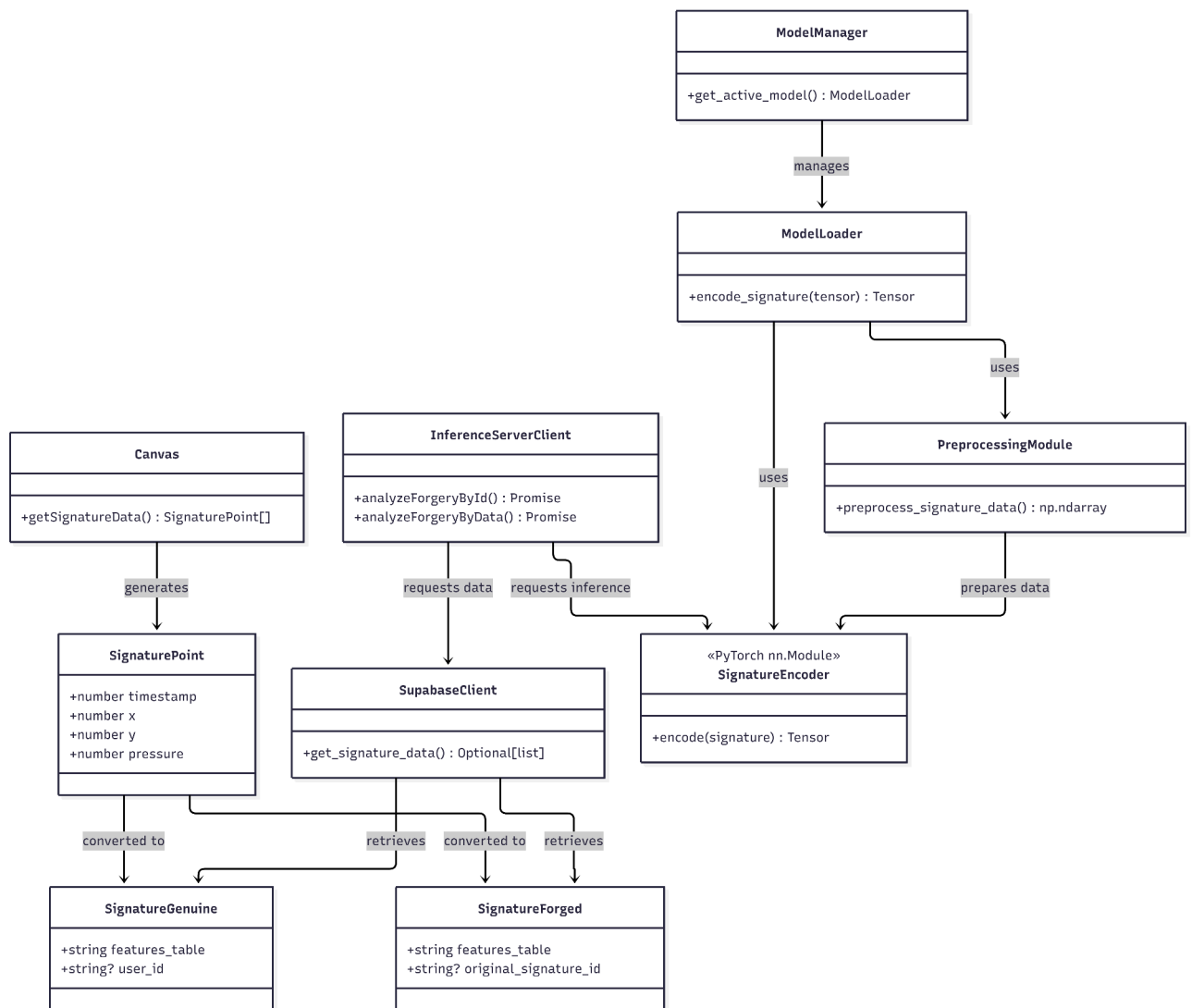


Рисунок 1.2 — Общая диаграмма объектов

Детальная диаграмма классов Frontend показывает структуру компонентов пользовательского интерфейса и типы данных. Данная диаграмма изображена на рисунке 1.3.

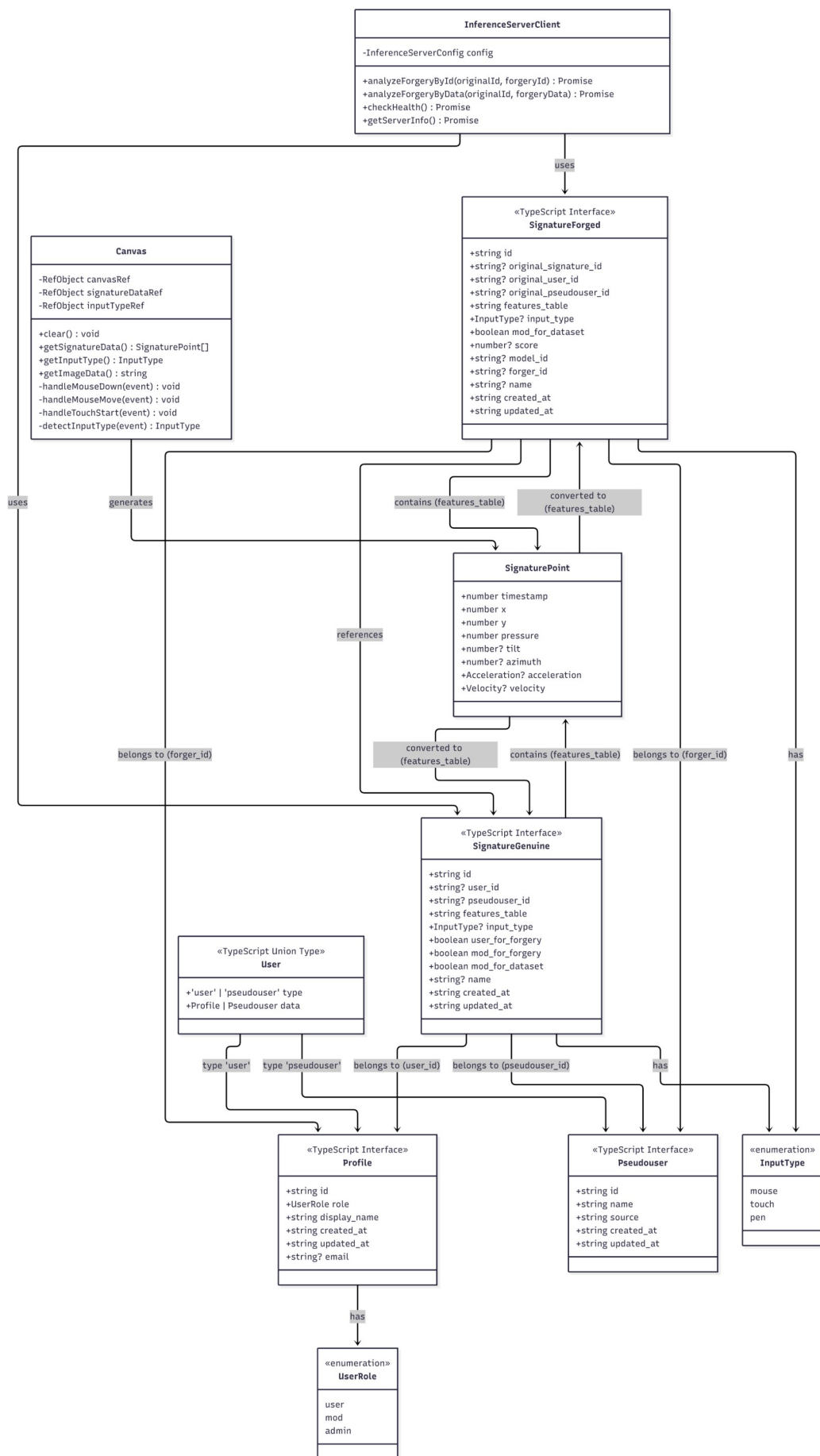


Рисунок 1.3 — Диаграмма классов Frontend

Описание Frontend классов приведены в таблице 1.1.

Таблица 1.1 — Описание Frontend классов

Класс	Описание
Canvas	React компонент для захвата цифровой подписи через HTML5 Canvas, обрабатывает события мыши, касания и стилуса, генерирует массив точек подписи
SignaturePoint	Структура данных для представления одной точки подписи (координаты, время, давление, опциональные данные о наклоне и ускорении)
InferenceServerClient	Клиент для взаимодействия с Backend inference сервером, отправляет запросы на анализ подделок и проверку состояния сервера
User	Объединенный тип (union type) для представления пользователя системы, может быть либо Profile, либо Pseudouser
Profile	Интерфейс данных пользователя с аутентификацией, содержит роль (user, mod, admin), отображаемое имя и email
Pseudouser	Интерфейс данных псевдопользователя из внешних источников, используется для работы с подписями из датасетов
SignatureGenuine	Интерфейс подлинной подписи из базы данных, содержит CSV данные подписи, метаданные и флаги для использования в обучении
SignatureForged	Интерфейс поддельной подписи, содержит ссылку на оригинальную подпись, оценку схожести и данные подделки
UserRole	Перечисление ролей пользователей (user, mod, admin) для системы авторизации
InputType	Перечисление типов ввода подписи (mouse, touch, pen) для классификации способа создания подписи

Детальная диаграмма классов Backend показывает структуру компонентов inference сервера и обработки данных. Данная диаграмма изображена на рисунке 1.4.

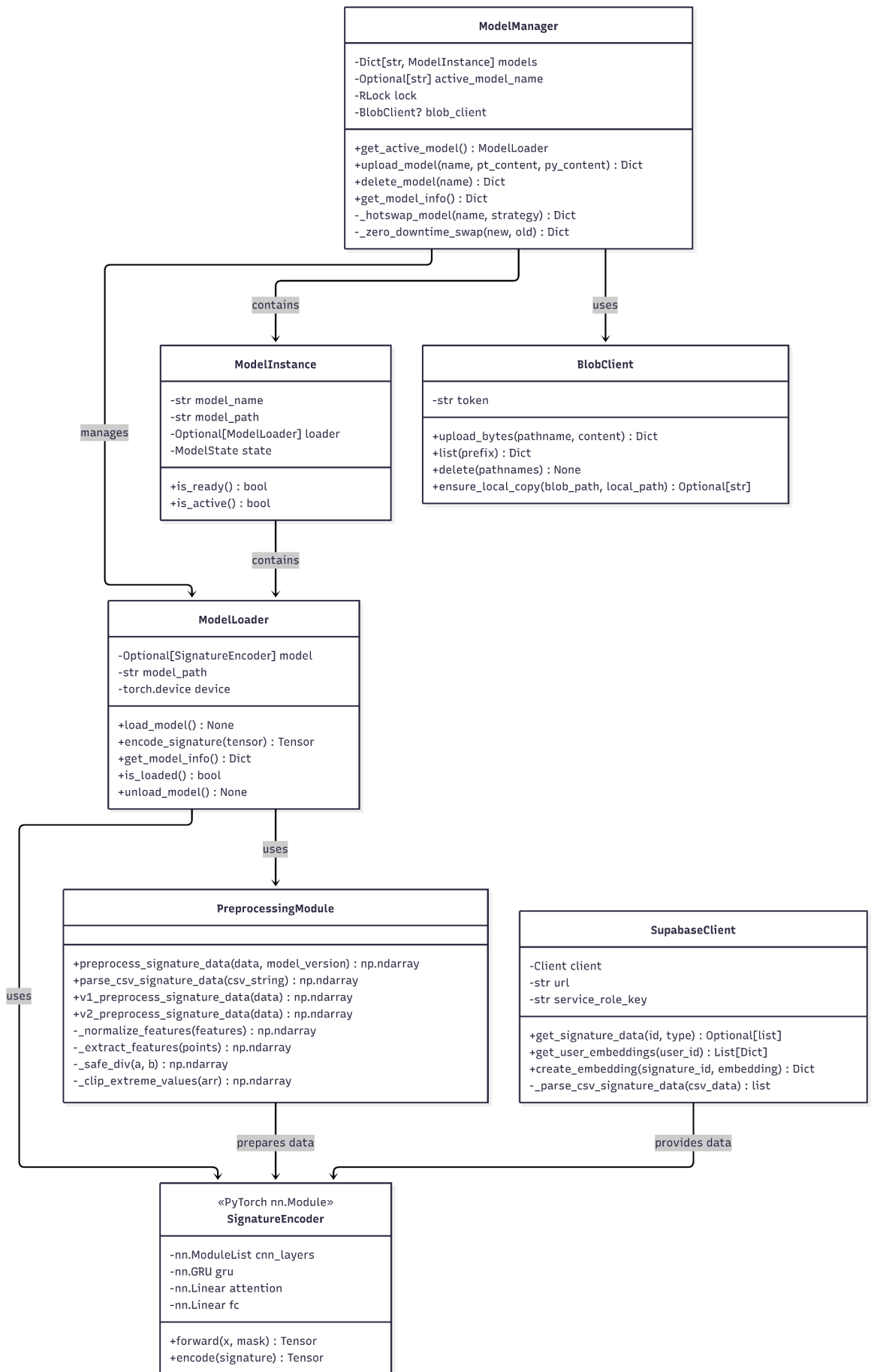


Рисунок 1.4 — Диаграмма классов Backend

Описание Backend классов приведены в таблице 1.2.

Таблица 1.2 — Описание Backend классов

Класс	Описание
SignatureEncoder	Нейронная сеть на PyTorch для кодирования подписей в векторные эмбединги.
ModelLoader	Класс для загрузки и управления PyTorch моделями, выполняет inference через SignatureEncoder, управляет памятью и устройством (CPU/GPU)
PreprocessingModule	Модуль предобработки данных подписей, выполняет нормализацию координат, вычисление признаков (скорость, ускорение, кривизна).
SupabaseClient	Клиент для работы с базой данных Supabase, использует service_role ключ для полного доступа, получает данные подписей и эмбедингов
ModelManager	Менеджер жизненного цикла ML моделей, поддерживает hotswap (замена моделей без остановки сервера), управляет загрузкой и активацией моделей
ModelInstance	Экземпляр загруженной модели с метаданными, отслеживает состояние модели (loading, ready, active, error) и путь к файлам модели
BlobClient	Клиент для работы с Vercel Blob Storage, загружает и синхронизирует файлы моделей между облачным хранилищем и локальным сервером

Дополнительные UML диаграммы классов находятся в Приложении Б.

Диаграммы последовательностей (Sequence) показывают последовательность взаимодействий между компонентами при выполнении основных операций (Рисунок 1.5, Рисунок 1.6):

- Создание подписи: User → Frontend → API → Auth → Database
- Верификация подписи: User → Frontend → Inference API → Database → Model → Preprocessing → Result
- Загрузка модели: Admin → Frontend → API → Blob Storage → ModelManager → ModelLoader

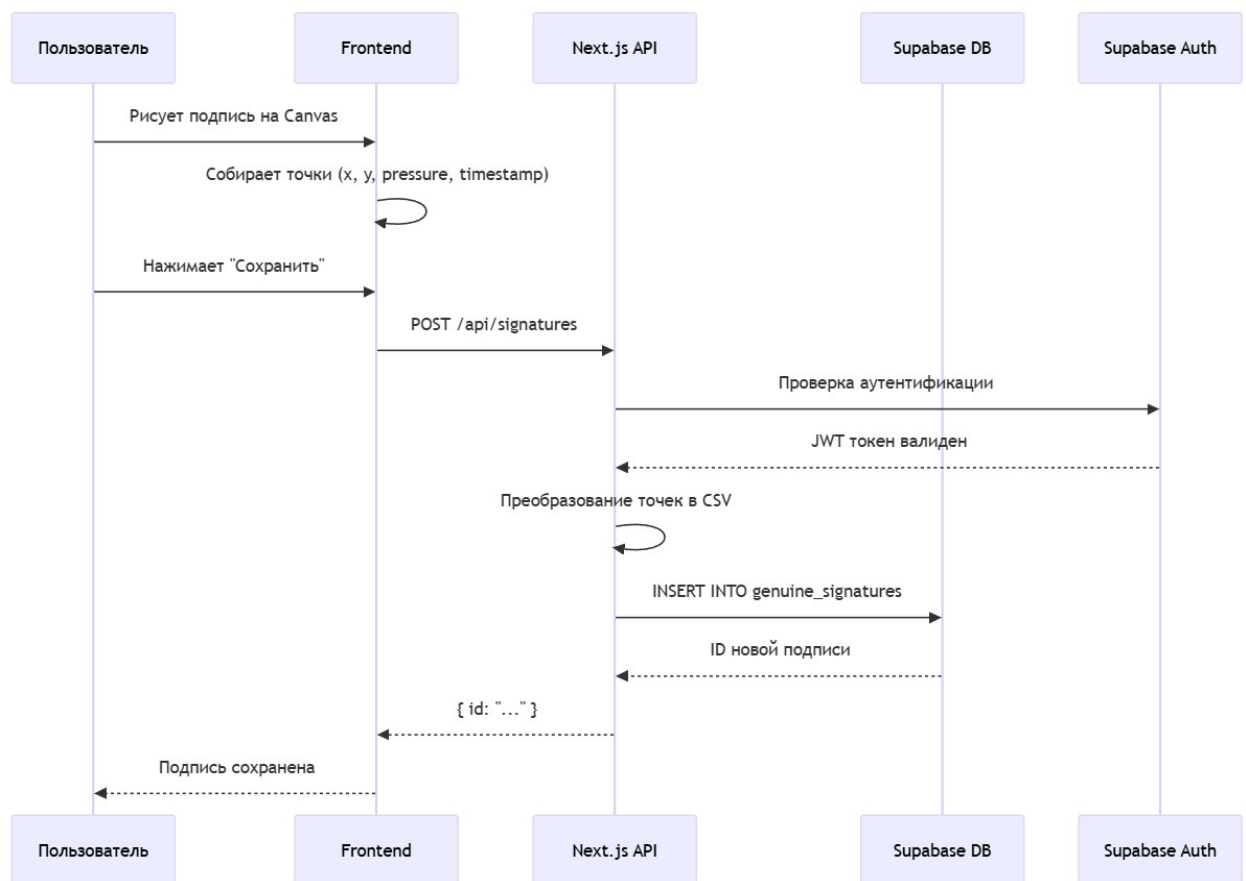


Рисунок 1.5 — Последовательность создания подписи

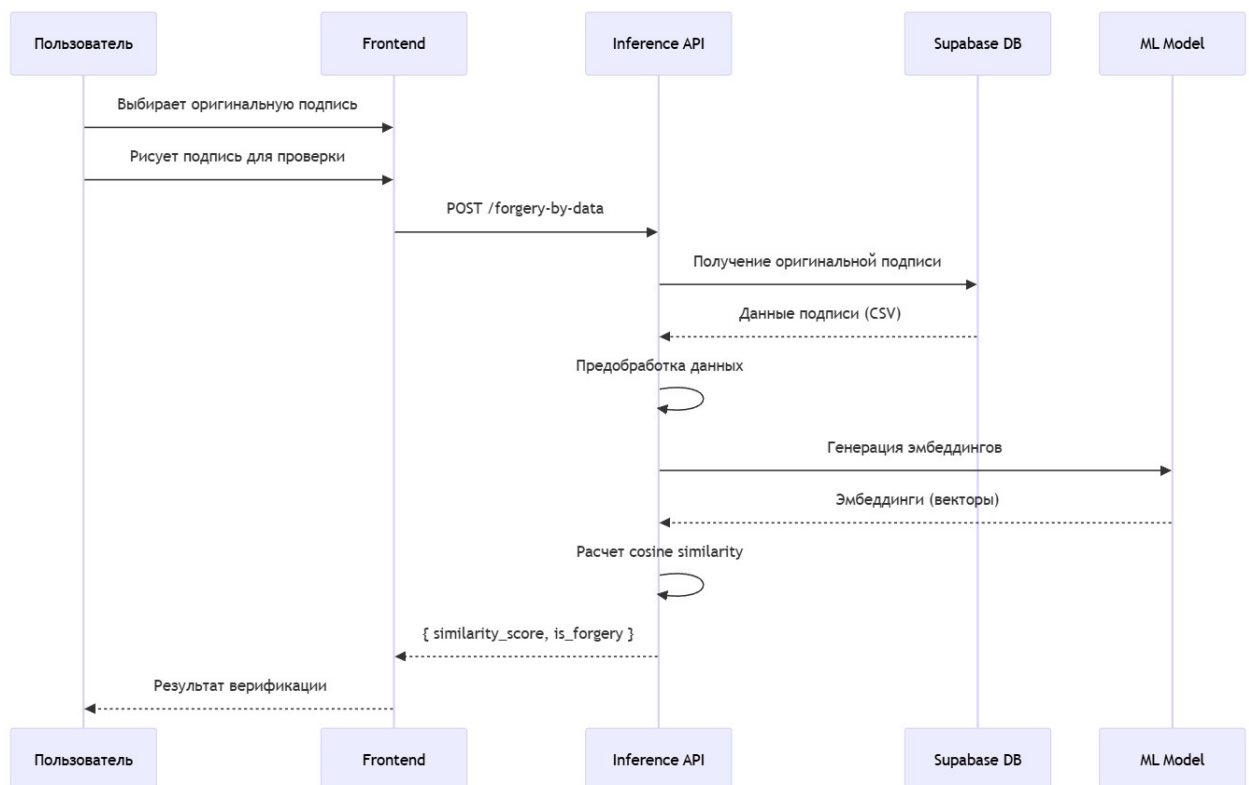


Рисунок 1.6 — Последовательность верификации подписи

Дополнительные диаграммы представлены в Приложении В.

1.3 Описание использованных в разработке компонентов

При разработке приложения использовались современные технологии и фреймворки, обеспечивающие высокую производительность, масштабируемость и удобство разработки. Все компоненты выбраны с учетом требований к сетевому программированию и работы с машинным обучением.

REST (Representational State Transfer) — архитектурный стиль для проектирования веб-сервисов, который основан на принципах обмена данными между клиентом и сервером через стандартные HTTP-методы (GET, POST, PUT, DELETE) для работы с ресурсами, обеспечивая масштабируемость, гибкость и отсутствие состояния (stateless). Применяется:

Ресурсно-ориентированные URL

- Использование HTTP методов для операций.
- JSON формат для обмена данными.
- Stateless взаимодействие (каждый запрос независим).

Все API endpoints следуют принципам REST:

/api/signatures - работа с подписями;

/forgery-by-id - верификация по ID;

/forgery-by-data - верификация по данным;

/health - проверка состояния сервера;

И тому подобные маршруты, используемые в Frontend Next.js App Router, Backend FastAPI сервере и базе данных Supabase.

Next.js — это React framework фреймворк, который предоставляет серверный рендеринг, статическую генерацию страниц и API routes. В проекте используется App Router — новая архитектура маршрутизации Next.js, которая обеспечивает:

- Серверные компоненты для улучшения производительности.

- Встроенную поддержку шаблонов (layouts) и вложенных путей (nested routes).

- API Routes для создания серверных эндпоинтов.

- Оптимизацию изображений и автоматическую оптимизацию кода.

Next.js API Routes используются для создания серверных эндпоинтов, которые обрабатывают запросы от клиента, выполняют валидацию данных и взаимодействуют с базой данных. Это позволяет реализовать серверную логику без необходимости отдельного backend сервера для некоторых операций.

React — библиотека для построения пользовательских интерфейсов. В проекте используется React 19 с поддержкой:

- Функциональных компонентов с хуками.

- Server Components для серверного рендеринга.

- Контекста для управления состоянием.

- Оптимизированного рендеринга.

React компоненты используются для создания интерактивного интерфейса, включая Canvas компонент для рисования подписей, формы аутентификации, списки подписей и административные панели.

TypeScript — язык программирования, типизированное надмножество JavaScript, которое обеспечивает:

- Статическую типизацию для выявления ошибок на этапе разработки.

- Автодополнение и улучшенную поддержку IDE.

- Рефакторинг кода с гарантией безопасности типов.

- Документирование интерфейсов через типы.

Все компоненты frontend написаны на TypeScript с определением типов для данных подписей, пользователей, API запросов и ответов.

Supabase Auth — система аутентификации, построенная на базе PostgreSQL и использующая JWT (JSON Web Tokens). Обеспечивает:

- Регистрацию и вход пользователей.

- Управление сессиями через JWT токены.
- Интеграцию с Row Level Security в PostgreSQL.
- Поддержку различных провайдеров аутентификации.

Аутентификация реализована через Supabase Auth Client, который автоматически управляет JWT токенами и их обновлением. Токены передаются через HTTP cookies для безопасности.

Tailwind CSS — utility-first CSS фреймворк, который позволяет быстро создавать интерфейсы с помощью predefined классов. Используется для:

- Стилизации всех компонентов интерфейса.
- Адаптивного дизайна.
- Темной темы (через next-themes).
- Кастомизации через конфигурационный файл.

FastAPI — современный веб-фреймворк для создания API на Python, основанный на стандартах OpenAPI и JSON Schema. Преимущества:

- Высокая производительность (сопоставима с Node.js и Go).
- Автоматическая генерация OpenAPI документации.
- Валидация данных через Pydantic.
- Асинхронная поддержка через async/await.
- Встроенная поддержка CORS, зависимостей, middleware.

FastAPI используется для создания REST API endpoints для верификации подписей.

PyTorch — фреймворк для машинного обучения, используемый для:

- Загрузки обученных моделей в формате .pt.
- Выполнения inference (предсказаний) на новых данных.
- Генерации эмбеддингов подписей.
- Работы с тензорами и нейронными сетями.

В проекте используется CPU-only версия PyTorch для совместимости с различными платформами развертывания. Модели загружаются в память при старте сервера и остаются активными для быстрого выполнения inference.

Uvicorn — ASGI (Asynchronous Server Gateway Interface) сервер для запуска FastAPI приложений. Обеспечивает:

- Асинхронную обработку запросов.
- Высокую производительность.
- Поддержку WebSocket (если необходимо).
- Hot reload для разработки.

Supabase Python Client — официальный клиент для работы с Supabase из Python приложений. Используется для:

- Выполнения SQL запросов к PostgreSQL.
- Работы с Row Level Security политиками.
- Получения данных подписей из базы данных.
- Использования Service Role ключа для обхода RLS.

PostgreSQL - реляционная база данных, используемая через Supabase.

Особенности:

- Расширение uuid-ossp для генерации UUID.
- Row Level Security (RLS) для безопасности на уровне строк.
- Триггеры для автоматического обновления метаданных.
- Индексы для оптимизации запросов.

RLS — механизм безопасности PostgreSQL, который позволяет контролировать доступ к строкам таблиц на основе политик. В проекте используются политики:

- Пользователи видят только свои подписи.
- Модераторы видят все подписи.
- Администраторы имеют полный доступ.
- Service Role ключ обходит RLS для backend операций.

Vercel Blob Storage — облачное хранилище для файлов, используемое для:

- Хранения ML моделей в формате .pt.
- Хранения определений архитектур моделей в формате .py.
- Доступа к моделям через HTTP URL.
- Синхронизации моделей между серверами.
- Хранение файлов любого другого формата, если необходимо.

Процесс загрузки моделей в Blob Storage:

- Администратор загружает файлы модели (.pt и .py) через веб-интерфейс Frontend (Next.js).
- Frontend отправляет файлы на Backend (FastAPI) через POST /model/upload эндпоинт.
- Backend сохраняет файлы локально в папку models/, если настроена соответствующая переменная окружения или локальная разработка.
- Backend автоматически загружает файлы в Vercel Blob Storage, если в production окружении.
- При запуске/рестарте, Backend сервер синхронизирует список моделей из Blob Storage и сохраняет их локально для быстрого доступа (при настроенной переменной окружения или при локальной разработке).

Vercel (vercel.com) — платформа для развертывания frontend приложений, обеспечивающая:

- Автоматическое развертывание при push в Git.
- Serverless функции для API Routes.
- CDN для статических ресурсов.
- Автоматический HTTPS.
- Аналитику и мониторинг.

Frontend приложение развертывается на Vercel с автоматической оптимизацией и масштабированием.

Render (render.com) — платформа для развертывания backend сервисов, используемая для:

- Развертывания FastAPI inference сервера.
- Постоянного сервера без cold starts (при платном хостинге).
- Автоматического масштабирования.
- Мониторинга и логирования.

Render выбран как основной вариант для inference сервера, Vercel имеет более жесткие бесплатные ограничения на проект.

Supabase Cloud — управляемый PostgreSQL хостинг, предоставляющий:

- Автоматические бэкапы.
- Масштабирование ресурсов.
- Мониторинг производительности.
- Интеграцию с другими сервисами Supabase.

Стоит также отметить глубокую интеграцию Vercel с Supabase при развертывании проекта, которая позволяет с легкостью создать базу данных с настроенным подключением для frontend приложения.

Zod — библиотека для валидации схем в TypeScript, используется для валидации данных на Frontend и в API Routes.

Pydantic — библиотека для валидации данных в Python, используется в FastAPI для валидации request/response моделей.

Radix UI — библиотека доступных UI компонентов для React, используется для создания доступного интерфейса.

Все перечисленные компоненты интегрированы в единую систему, обеспечивающую надежную работу приложения для верификации цифровых подписей с использованием машинного обучения.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Руководство пользователя

При открытии главной страницы сайта, неавторизованный пользователь увидит интерфейс подделки подписи. Здесь он может попробовать подделать случайно выданную подпись.

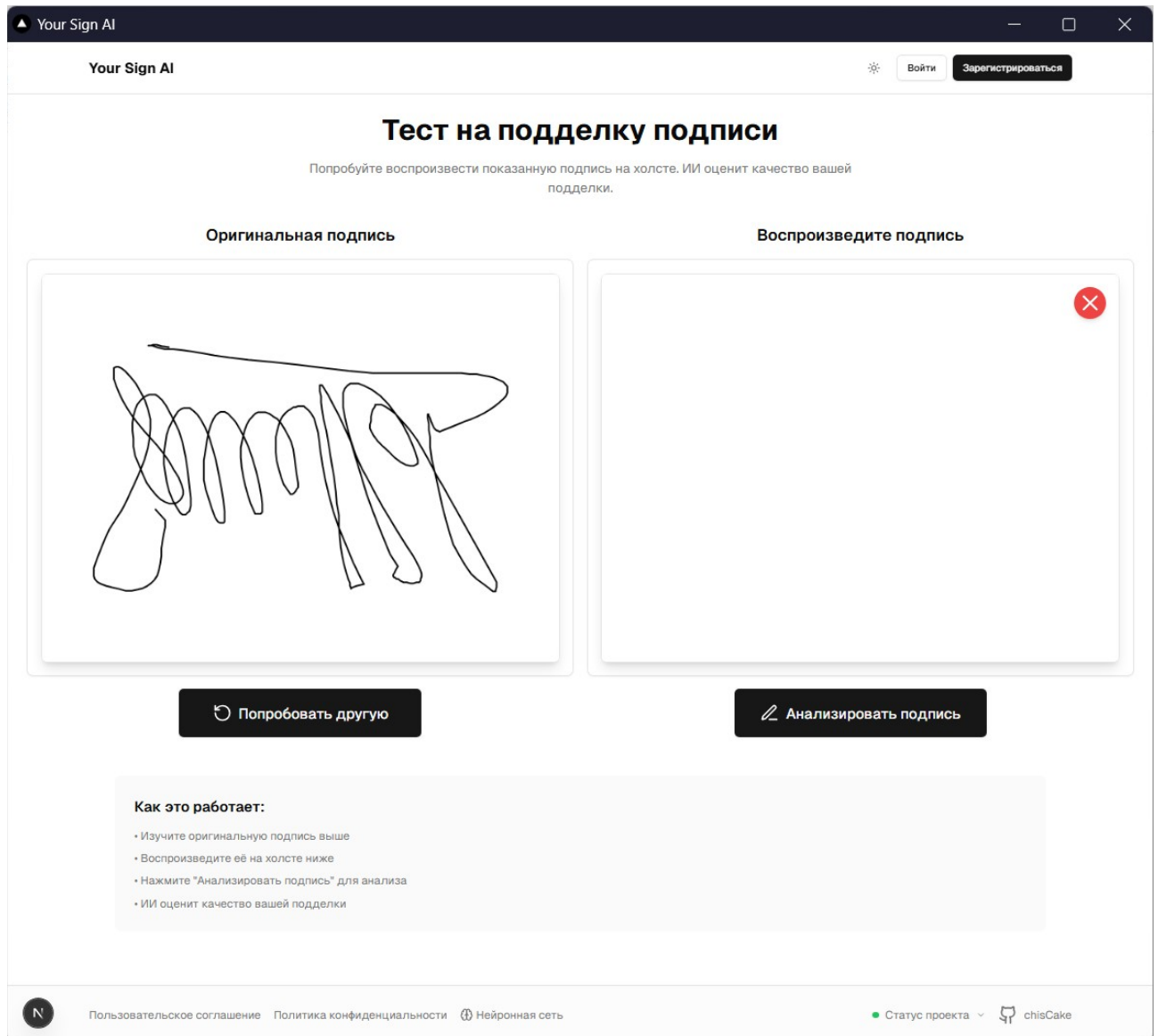


Рисунок 2.1 — Главная страница

Пример попытки подделки подписи можно увидеть на рисунке 2.2 и результат анализа на рисунке 2.3.

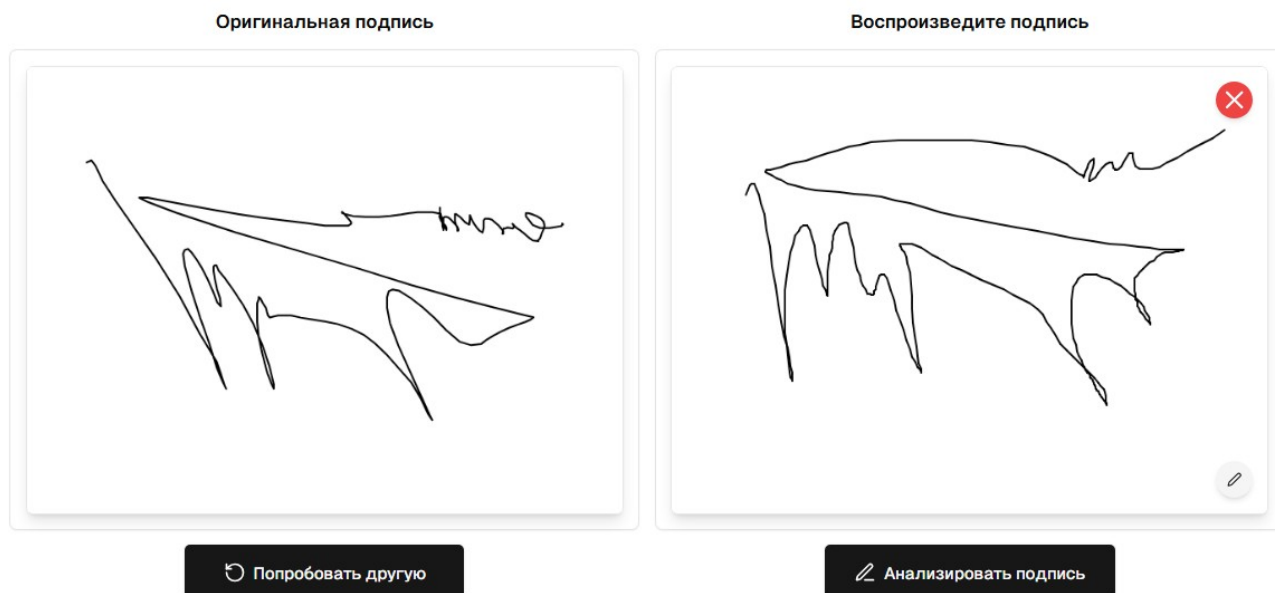


Рисунок 2.2 — Подделка подписи

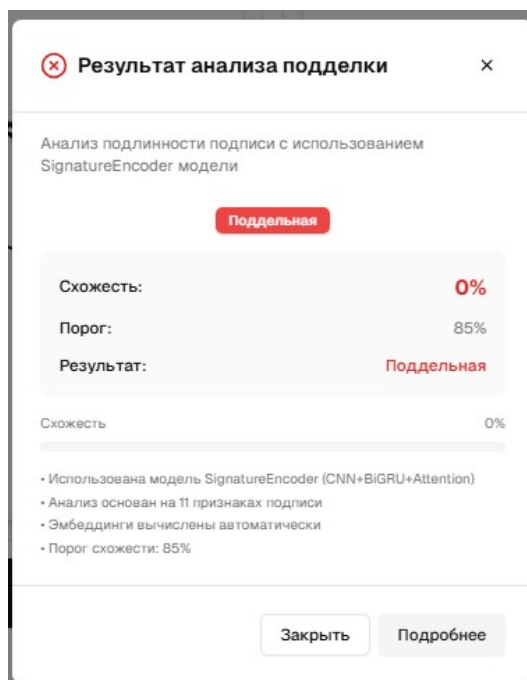


Рисунок 2.3 — Результат анализа подделки

Забегая вперед, анализ настоящей подписи, т. е. подпись подделывает ее создатель, на странице обзора подписей, изображены на рисунках 2.4 и 2.5.

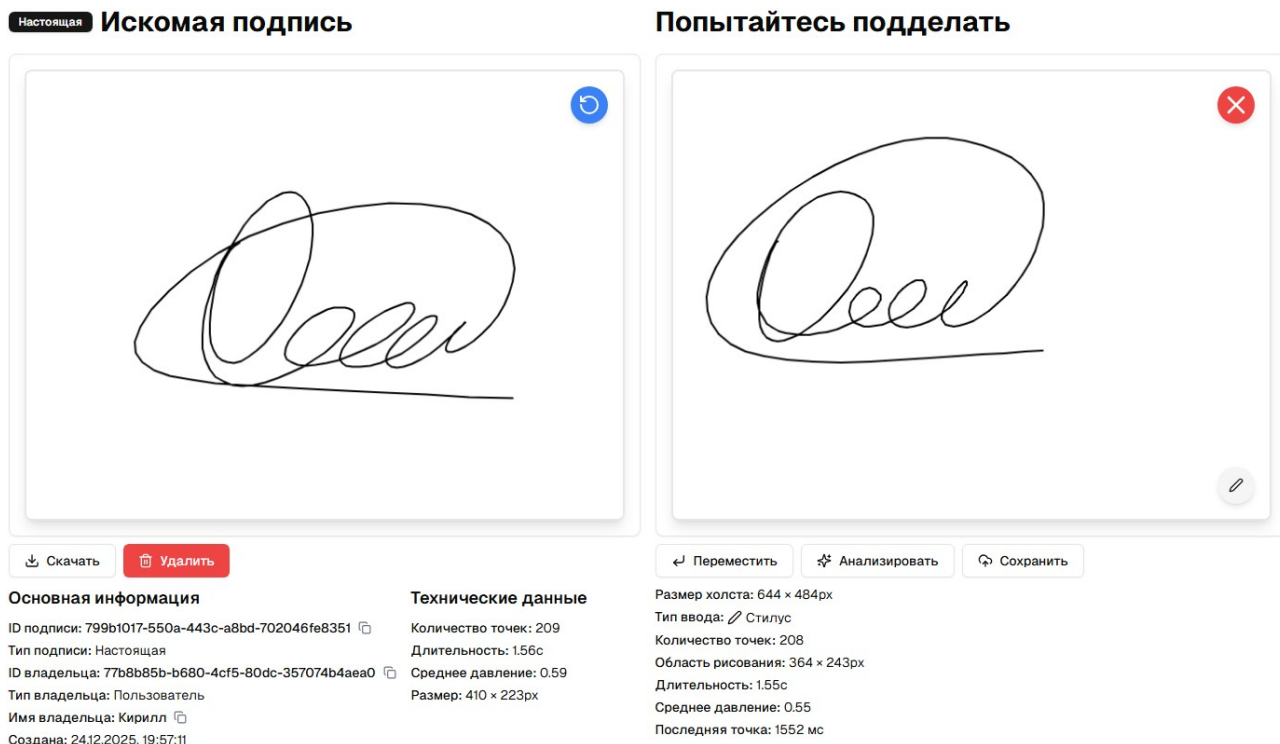


Рисунок 2.4 — Воспроизведение собственной подписи

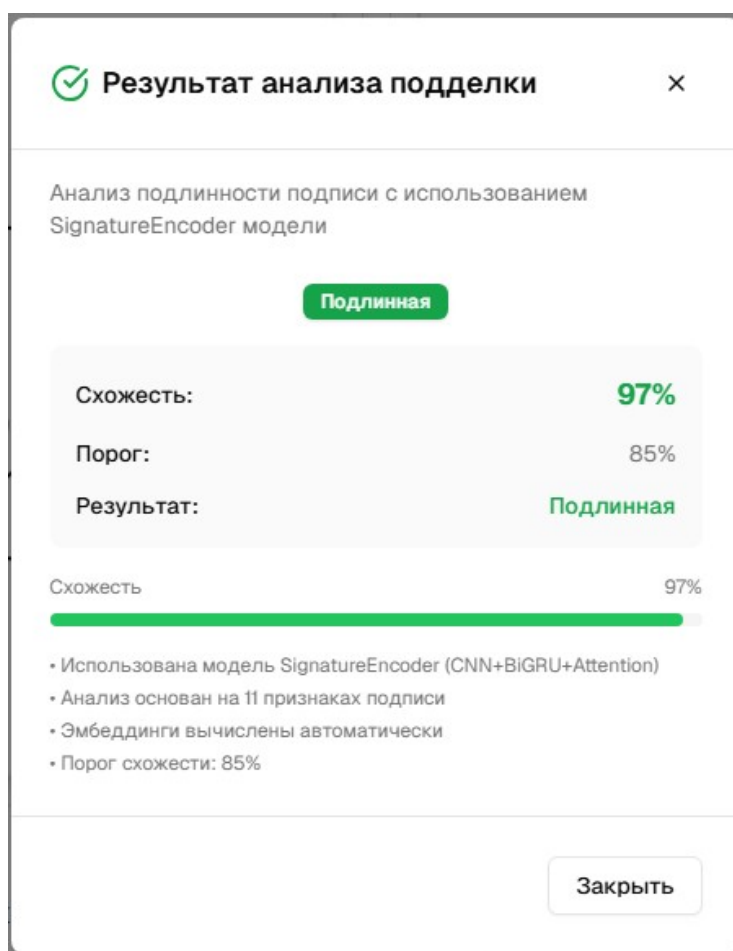


Рисунок 2.5 — Результат анализа подписи

Авторизованным пользователем доступен личный кабинет, изображенный на рисунках 2.6 и 2.7. В нем можно посмотреть информацию о своем аккаунте, добавить новую подпись и управлять уже сохраненными. Под «управлением» подразумевается следующий функционал:

- Просмотр детальной характеристики (как на рисунке 2.4).
- Скачивание подписей как изображений.
- Включение и отключение опции участия подписи в публичной выборке как экземпляра для подделки.

Личный кабинет

Управляйте своими подписями и настройками

Информация о профиле

Имя	Роль
Кирилл	admin
ID	
fe12d9d5-4bc8-4197-bd96-e71763c2e938	
Email	
kirilloleshkevich7@gmail.com	
Дата регистрации	
8 октября 2025 г.	

Создать подпись

☐ Разрешить использование как пример для подделки

Сохранить

Рисунок 2.6 — Личный кабинет, информация и холст для добавления подписи

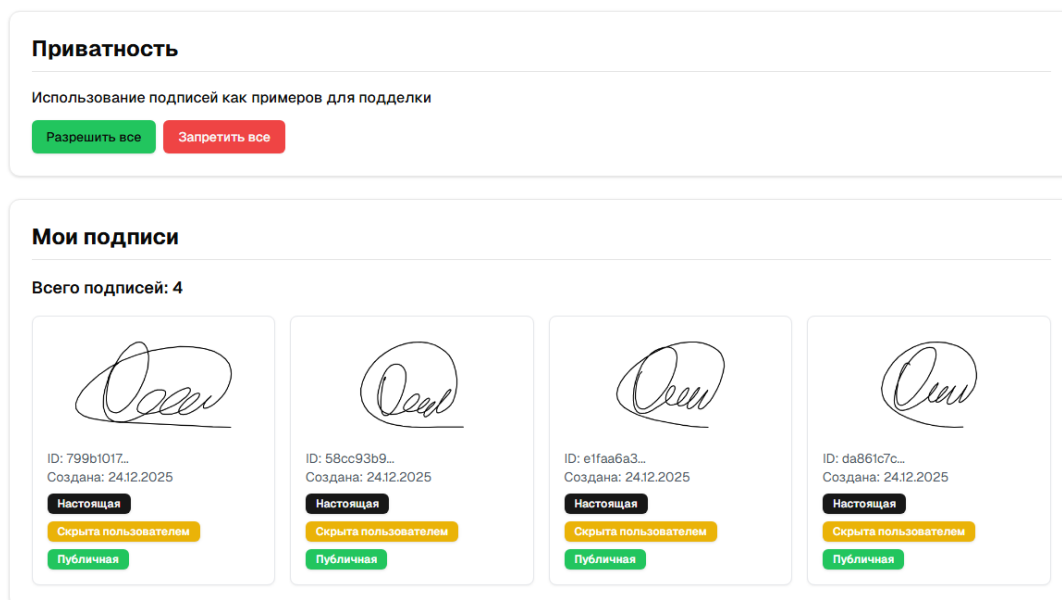


Рисунок 2.7 — Личный кабинет, настройки видимости и список подписей

При выборе подписи, открывается модальное окно для быстрого обзора, как показано на рисунке 2.8. Здесь пользователь может открыть подпись в новой странице, скачать, удалить и посмотреть ее воспроизведение во времени с помощью специальной кнопки в верхнем-правом углу холста.

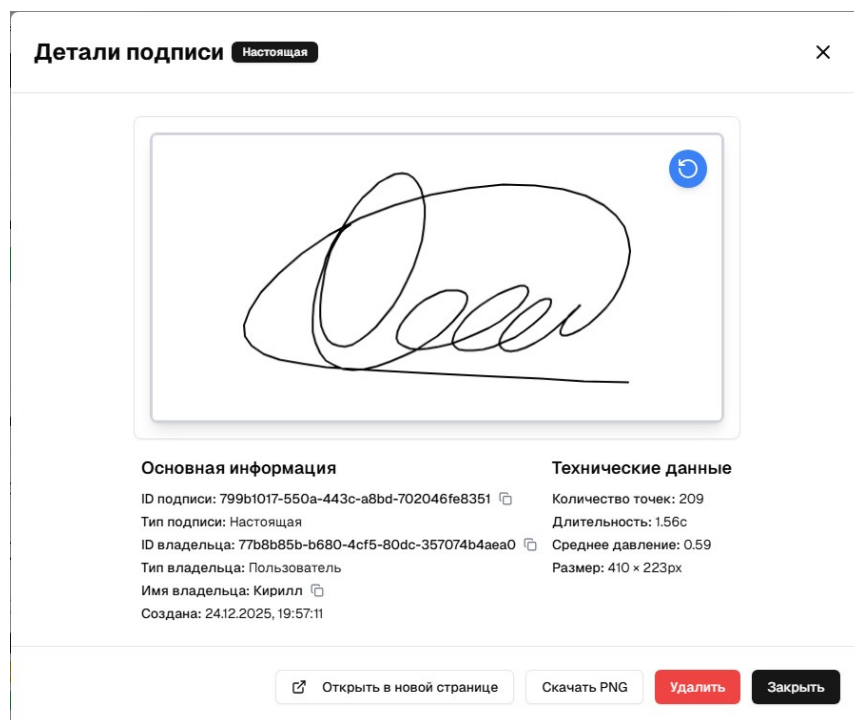


Рисунок 2.8 — Модальное окно выбранной подписи

Для авторизованных пользователей, которые имеют роль модератора, представляется новый функционал в виде:

- Полного доступа к имеющимся подписям.
- Статистика по подписям и пользователям.
- Возможность добавления подписей от имени псевдопользователей.
- Просмотр информации о профилях.

Тут стоит обратить внимание на концепцию псевдопользователей: модератор может создать новый аккаунт, который не будет считаться реальным аккаунтом и не будет связан с Supabase Auth. Такой профиль нужен для структурированной сборки датасета, если пользователь не регистрируется. Пример ситуации для использования — сбор подписей людей на каком-либо мероприятии, без траты времени на полноценную регистрацию.

На рисунке 2.9 изображена панель модератора, где можно увидеть сводку по подписям и пользователям, а также несколько последних добавленных подписей.

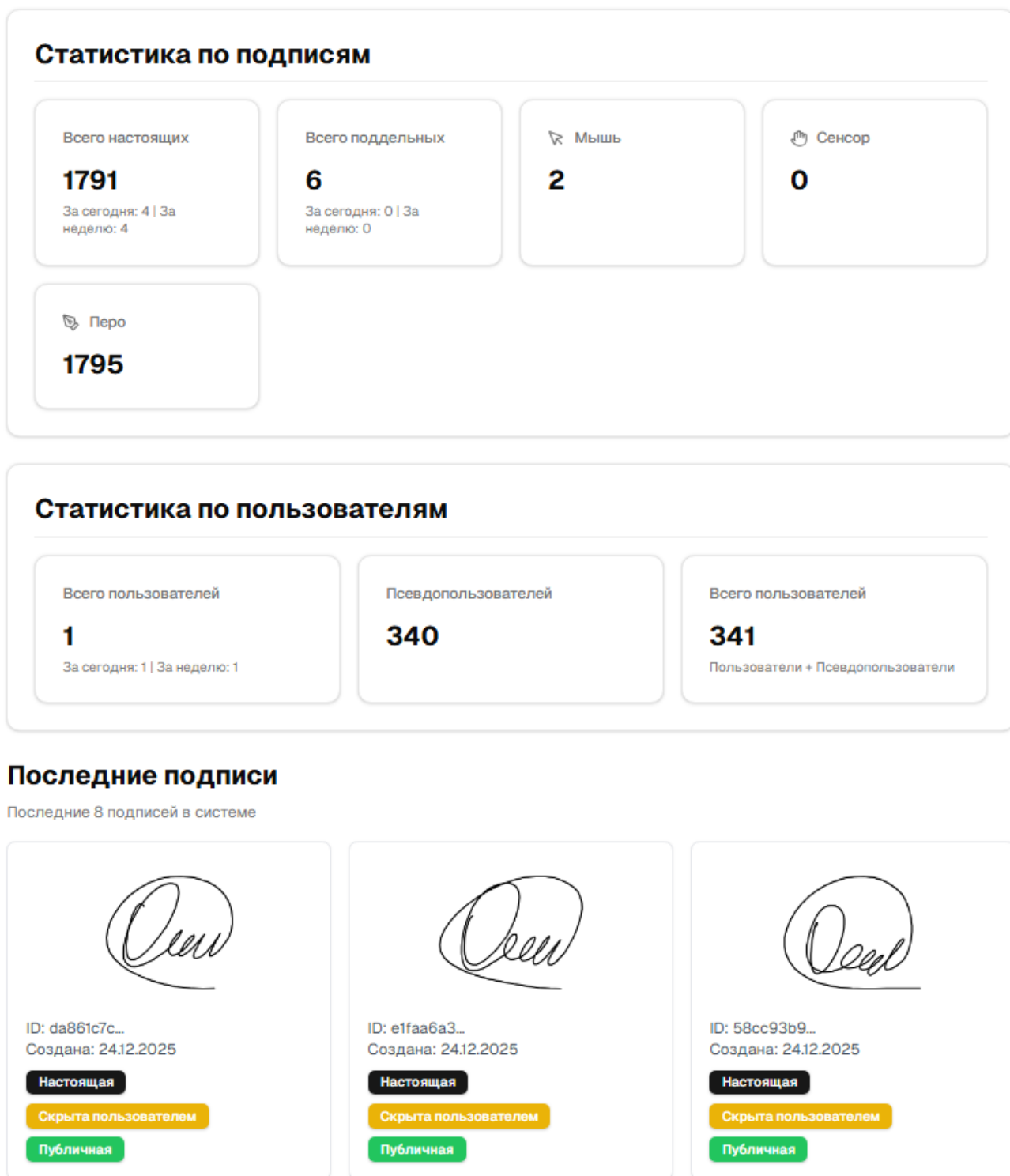


Рисунок 2.9 — Панель модератора

На рисунке 2.10 изображена страница со всеми подписями. Здесь модератор может найти нужную подпись по ее идентификатору или отфильтровать по заданным параметрам, например настоящая/поддельная или по дате добавления.

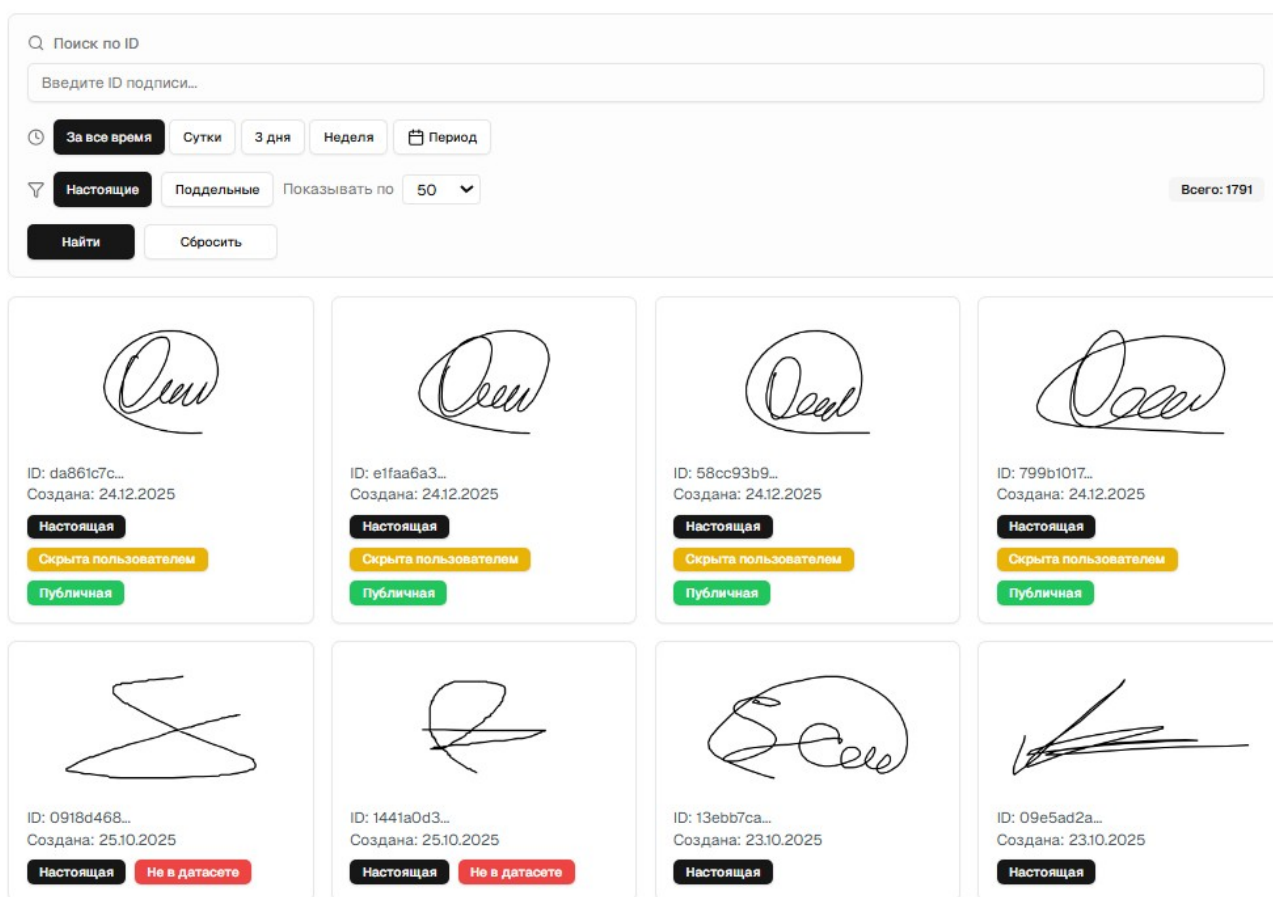


Рисунок 2.10 — Страница обзора подписей

На рисунке 2.11 изображена страница со всеми пользователями. Здесь модератор может найти пользователя по имени, отфильтровать по их типу (обычные или псевдопользователи), указать источник для псевдопользователей — это полезно для поиска пользователей из разных внешних датасетов или мест добавления.

Управление пользователями

Просмотр и управление пользователями системы и их подписями

Пользователи

Выберите пользователя для просмотра подробной информации

Фильтры

- ☒ Обычные пользователи
- ☒ Псевдопользователи

Поиск

Источник (псевдо)

☒ Включая

Найдено: 345

Кирилл

Пользователь admin

Модератор

Пользователь mod

Модератор

Пользователь mod

Админ

Пользователь admin

Пользователь

Пользователь user

Информация о пользователе

Имя	u0263	Тип	Псевдопользователь
ID	00c8df6e-ef46-4ad8-9906-27e9a48a5394	Дата создания	7 октября 2025 г. в 22:40
Источник	dsdb		

Управление

Управление настройками и данными пользователя

☐ Использование подписей как примеров для подделки

☒ Использование подписей в датасете

☒ Профиль

Подписи пользователя

Все подписи, созданные этим пользователем

Всего подписей: 25

ID: 05897528...
Тип ввода: rep
Дата создания: 07.10.2025

ID: 0c480ed8...
Тип ввода: rep
Дата создания: 07.10.2025

ID: 18052f5e...
Тип ввода: rep
Дата создания: 07.10.2025

ID: 315bc15d...
Тип ввода: rep
Дата создания: 07.10.2025

Рисунок 2.11 — Страница обзора пользователей

На рисунке 2.12 изображена страница контролируемого добавления — здесь модератор может найти или создать нового псевдопользователя и дать человеку, в его присутствии, занести его настоящие подписи в базу данных. Процесс добавления подписей выглядит следующим образом: модератор создает нового псевдопользователя, передает управление другому человеку, и этот человек начинает добавлять свои подписи (как в личном кабинете), под присмотром модератора.

Выбор пользователя

Введите имя пользователя

Иванов И. И.

Нет результатов

Создать

Пользователь создан

Иванов И. И.

Псевдо

ID: f8ca99e4-3323-43ba-870b-fd4f22da472a

Дата создания: 15.12.2025

Источник: manual

Начать

Подписи пользователя

Нет подписей для отображения

Рисунок 2.12 — Страница контролируемого добавления

Панель администратора содержит функции для управления работающей нейросетевой моделью. Просмотр текущего состояние inference сервера через API запрос (рисунок 2.13), управление моделями в наличии (рисунок 2.14) и добавление новой модели (2.15).

Панель администратора

Управление системой и настройками

ИИ Сервер

Состояние сервера

● Работает нормально

Модель:

v1 (cpu)

Память:

75.09 MB

Supabase:

Подключен

Последняя проверка: 16:52:26

<http://localhost:8000/health>

Рисунок 2.13 — Панель администратора. Состояние inference сервера

В секции «Статус моделей» (рисунок 2.14) администратор может произвести замену работающей на inference сервере модели или удалить какую-либо модель. Здесь также может увидеть какие модели хранятся в Vercel Blob, а какие на inference сервере.

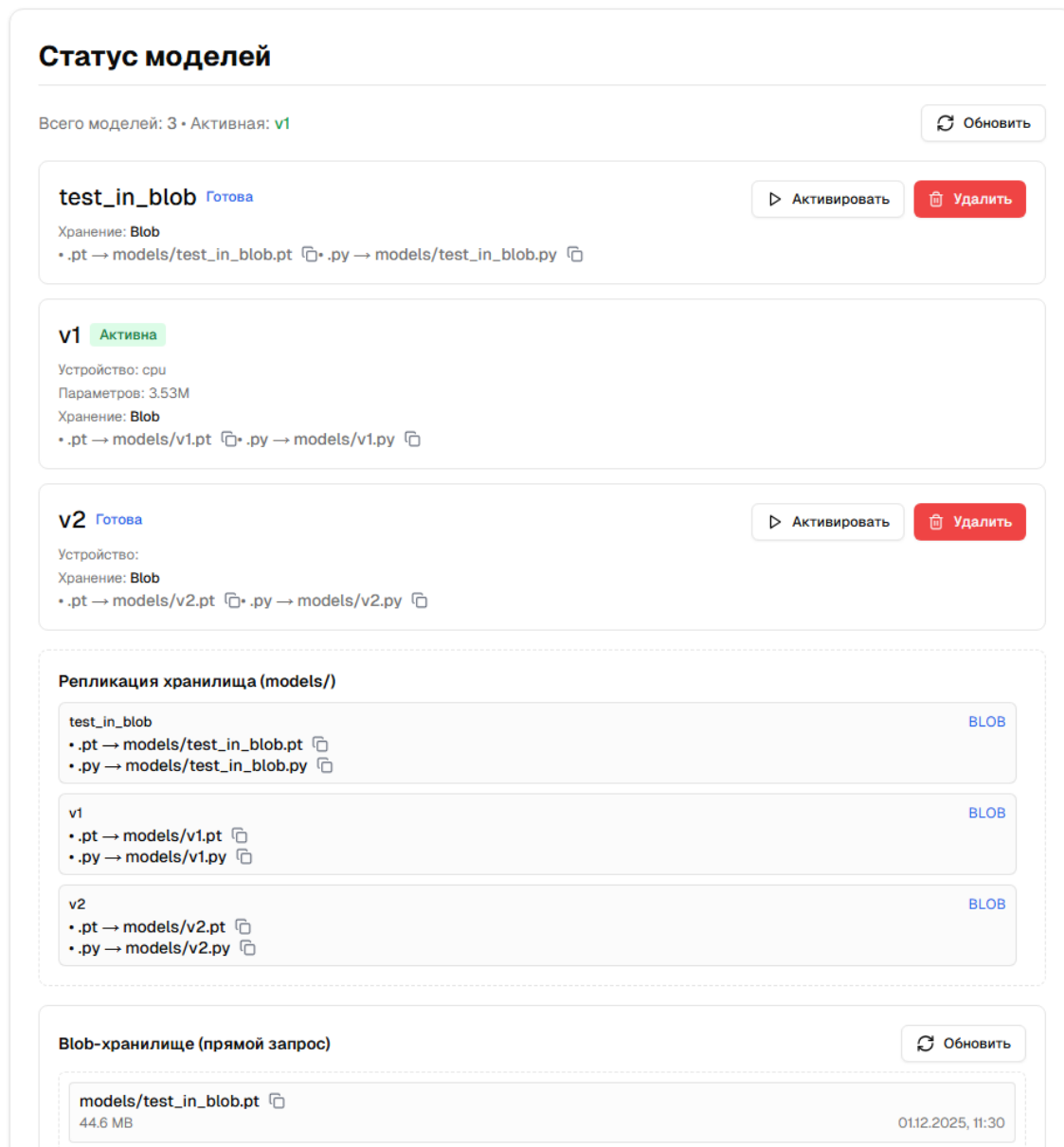


Рисунок 2.14 — Панель администратора. Статусы моделей в наличии

В секции «Загрузка моделей» (рисунок 2.15) администратор может сохранить новую обученную модель загрузив файл весов модели .pt и файл кода модели .py. Также можно выбрать стратегию замены: «горячая» замена позволяет пустить модель в использование, но отрицательной стороной является возможность нехватки памяти (особенно на бесплатном хостинге).

Последовательная замена решает эту проблему, но может ограничить запросы на анализ подписей в этот период.

Загрузка модели

Текущее окружение

DEVELOPMENT

Файлы сохраняются только на локальном диске inference-сервера.

Имя модели

например: v3

Только буквы, цифры, дефисы и подчеркивания

Файл весов модели (.pt)

Выберите файл Файл не выбран

Файл кода модели (.py)

Выберите файл Файл не выбран

Стратегия замены

☒ Zero Downtime

Без простоя: новая модель запускается параллельно

☐ Sequential

Последовательно: старая останавливается, затем запускается новая

Важно:

Файлы .pt (веса) и .py (архитектура) должны быть совместимы. Если модель не загрузится, проверьте, что архитектура в .py соответствует обученным весам в .pt.

Загрузить модель

Zero Downtime:

Старая модель продолжит работать во время загрузки новой. После успешной загрузки новая модель активируется, а старая будет остановлена.

Рисунок 2.15 — Страница контролируемого добавления

Управление подписью как со стороны обычных пользователей, так со стороны модераторов или администраторов может происходить через простой интерфейс при наведении на превью подписи. Причем набор доступных кнопок отличается между ролью и принадлежностью подписи. На рисунке 2.16 приведен пример подписи, которую просматривает администратор и является ее владельцем (т. е. имеет полный набор действий). Назначение кнопок слева-направо: открыть в новой странице, скачать как изображение, скрыть от лица пользователя, скрыть от лица модератора, убрать из участия при сборке датасета и удаление.

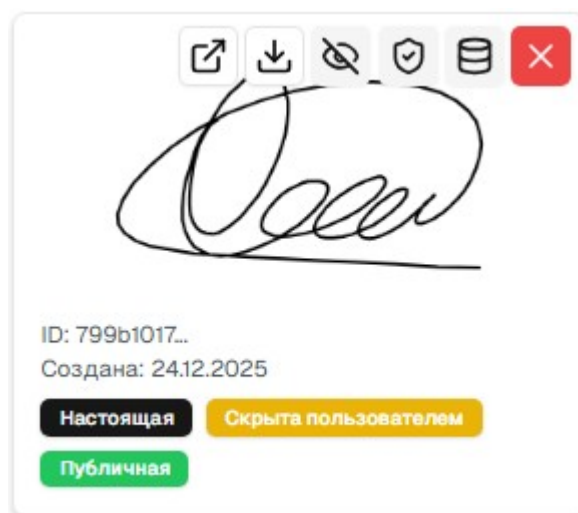


Рисунок 2.16 — Превью подписи с кнопками управления и ее метками-статусами

В разделе About на странице «Нейронная сеть» можно посмотреть статус работающей модели и ее характеристики вместе с кодом. Эта страница публичная и доступна для всех. Информация для этой страницы генерируется посредством API запроса на inference сервер и обновляется автоматически. Пример данной страницы изображен на рисунке 2.17.

Нейронная сеть

Обновить

☰

Состояние Inference сервера

● Работает нормально

Модель: v1 (cpu)

Память: 52.05 MB

Supabase: Подключен

Последняя проверка: 17:10:44

ⓘ

Информация о модели

Скрыть

Модуль: models.v1

Тип модели: SignatureEncoder

Устройство: cpu

Статус: Загружена

Всего параметров: 3 528 961

Обучаемых параметров: 3 528 961

Архитектура:

CNN(1D) → BiGRU → Attention → FC → L2-normalized embedding

Конфигурация модели:

```
{
  "name": "hybrid",
  "embedding_dim": 256,
  "gru_hidden": 256,
  "gru_layers": 3,
  "dropout": 0.2,
  "in_features": 11,
  "conv_channels": null
}
```

Конфигурация системы:

```
{
  "module": "models.v1",
  "class_name": "SignatureEncoder",
  "file_path": "models/v1.py",
  "checkpoint_path": "models/v1.pt"
}
```

<>

Исходный код модели

Скрыть

```
# src/models/hybrid.py
import torch
import torch.nn as nn
import torch.nn.functional as F
```

Рисунок 2.17 — Обзор информации о текущей рабочей нейросети

Вся архитектура обеспечивает полный цикл анализа, управления и поддержки моделей верификации подписей. Такой комплексный подход позволяет не только оперативно собирать и структурировать датасеты, но и гарантирует гибкость в обновлении и обслуживании ключевой нейросетевой модели.

2.2 Результаты тестирования программного продукта

Для обеспечения качества разработанного приложения был выбран комплексный подход к тестированию, включающий два основных инструмента:

Jest — фреймворк для unit-тестирования и тестирования компонентов. Jest выбран по следующим причинам:

- Интеграция с экосистемой React и Next.js — Jest является стандартным инструментом тестирования для React-приложений.
- Поддержка TypeScript из коробки — не требует дополнительной настройки для работы с TypeScript.
- Богатый набор функций — встроенные моки, snapshot-тестирование, покрытие кода.
- Быстрое выполнение тестов — оптимизирован для быстрого запуска unit-тестов.
- Интеграция с Testing Library — используется `@testing-library/react` и `@testing-library/jest-dom` для тестирования React-компонентов с фокусом на пользовательский опыт.

В проекте Jest настроен с использованием `jest-environment-jsdom` для эмуляции браузерного окружения, что позволяет тестировать React-компоненты без необходимости запуска реального браузера. Тесты покрывают логику утилит, хуков и компонентов, обеспечивая проверку корректности работы отдельных модулей системы.

Playwright — инструмент для end-to-end (E2E) тестирования. Playwright выбран по следующим причинам:

- Кросс-браузерное тестирование — поддержка Chromium, Firefox и WebKit из коробки.
- Надежность и стабильность — автоматическое ожидание элементов, встроенная обработка асинхронных операций.
- Мощные возможности — автоматические скриншоты, видео записи, трассировка выполнения тестов.

- Поддержка различных ролей пользователей — в проекте настроены отдельные проекты для тестирования функциональности гостей, пользователей, модераторов и администраторов.
- Интеграция с Next.js — возможность запуска dev-сервера автоматически перед тестами.
- Удобная настройка окружения — поддержка различных storage states для аутентификации разных ролей.

В проекте Playwright используется для тестирования полных пользовательских сценариев: аутентификация, создание подписей, верификация и т. п. Тесты организованы по ролям пользователей, что позволяет проверять корректность работы системы для каждой категории пользователей отдельно.

Комбинация Jest и Playwright обеспечивает многоуровневое тестирование: Jest покрывает unit-тесты и тесты компонентов, а Playwright - интеграционные и E2E тесты, что позволяет выявить проблемы на разных уровнях системы.

Так как Jest является фреймворком модульного тестирования, то тестирование кода происходит только для кода lib/ — отвечающего за слой бизнес логики веб-сайта, не требующего сложных взаимодействий со стороны представлений.

Также стоит отметить, что тестирование кода, связанного с базой данных, происходит с помощью локальной версии базы данных Supabase, которая предоставляет идентичный веб-интерфейсу функционал, но развернутый в Docker.

Результаты модульного Jest тестирования, изображенные на рисунке 2.18, показывают прохождение 511 тестов, которые покрывает >90% кода frontend части (Next.js).

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.8	86.57	93.13	95.73	
lib	97.29	82.85	89.13	100	
inference-client.ts	97.22	75	86.66	100	49,96,135,214-268
model-management-client.ts	92.3	70.58	62.5	100	74,100,120,153,171
signature-management-policies.ts	100	100	100	100	
types.ts	100	100	100	100	
lib/hooks	100	100	100	100	
use-page-title.ts	100	100	100	100	
use-signature.tsx	100	100	100	100	
use-user.ts	100	100	100	100	
lib/supabase	93.19	85.71	91.66	92.96	
client.ts	100	100	100	100	
middleware.ts	81.57	91.66	50	80	39-49,91
queries.ts	94.13	83.92	100	93.96	...,311,413,416,474-475,501-502,528-529,546-547
server.ts	100	100	100	100	
service.ts	100	100	100	100	
lib/utils	93.49	85.79	93.24	94.71	
auth-client-utils.ts	96.66	87.5	100	98.27	68
auth-server-utils.ts	100	100	100	100	
auth-utils.ts	100	100	100	100	
badge-factory.tsx	100	92.3	100	100	61
client-utils.ts	100	100	100	100	
mod-client-utils.ts	95.4	76.66	100	96.2	172-174
signature-utils.ts	89.2	82.75	86.11	90.76	311-313,333-338,348-350,375-380,390-392,437-439
user-utils.ts	100	100	100	100	
Test Suites: 20 passed, 20 total Tests: 511 passed, 511 total Snapshots: 0 total Time: 13.129 s					

Рисунок 2.18 — Таблица покрытия кода и результаты тестирования Jest

Для более сложного тестирования, имитирующего работу с интерфейсом со стороны пользователя, с помощью фреймворка Playwright были созданы и проведены множества тестов для взаимодействия непосредственно со страницами.

Тесты были разделены на роли: гостевые тесты, пользовательские, тесты модератора и администратора, а также на стадии: создание заготовленных аккаунтов, вход в аккаунты, выполнение тестов и автоматическое очистка (teardown) в конце.

Некоторые разделы и тесты изображены на рисунке 2.19.

Q	All 72	✓ Passed 72	Failed 0	Flaky 0	Skipped 0	⚙
15.12.2025, 18:15:35 Total time: 2.0m						
✓ setup/auth.setup.mts ✓ @admin login and save storage setup-admin admin 12.9s setup/auth.setup.mts:17 ✓ @mod login and save storage setup-mod mod 12.8s setup/auth.setup.mts:17 ✓ @user login and save storage setup-user user 12.9s setup/auth.setup.mts:17						
about/about-pages.spec.ts						
auth/login.spec.ts ✓ Login page › should display login form guest 1.5s auth/login.spec.ts:5 ✓ Login page › should show validation error for empty email guest 2.6s auth/login.spec.ts:17 ✓ Login page › should show validation error for empty password guest 2.3s auth/login.spec.ts:32 ✓ Login page › should show error for invalid credentials guest 2.1s auth/login.spec.ts:48 ✓ Login page › should successfully login with valid credentials guest 4.6s auth/login.spec.ts:64 ✓ Login page › should have link to sign-up page guest 2.5s auth/login.spec.ts:76						
auth/sign-up.spec.ts						
guest/home.spec.ts						
user/analyze-forgery.spec.ts ✓ Analyze Forgery (User) › should display forgery analysis interface user 7.1s user/analyze-forgery.spec.ts:15 ✓ Analyze Forgery (User) › should show error when trying to analyze empty signature user 7.3s user/analyze-forgery.spec.ts:37 ✓ Analyze Forgery (User) › should analyze forgery by drawing on canvas user 38.1s user/analyze-forgery.spec.ts:59 ✓ Analyze Forgery (User) › should display similarity score in result user 41.9s user/analyze-forgery.spec.ts:155 ✓ Analyze Forgery (User) › should allow getting new signature user 9.3s user/analyze-forgery.spec.ts:230 ✓ Analyze Forgery (User) › should handle analysis error gracefully user 13.7s user/analyze-forgery.spec.ts:249						
user/create-signature.spec.ts						
user/dashboard.spec.ts						
mod/manage-signatures.spec.ts ✓ Manage Signatures (Moderator) › should search signature by ID mod 8.7s mod/manage-signatures.spec.ts:15 ✓ Manage Signatures (Moderator) › should filter signatures by category mod 10.5s mod/manage-signatures.spec.ts:36 ✓ Manage Signatures (Moderator) › should change items per page mod 7.9s mod/manage-signatures.spec.ts:55 ✓ Manage Signatures (Moderator) › should navigate through pages mod 8.3s mod/manage-signatures.spec.ts:81 ✓ Manage Signatures (Moderator) › should reset filters mod 7.0s mod/manage-signatures.spec.ts:106 ✓ Manage Signatures (Moderator) › should display signature details when signature is clicked mod 8.8s mod/manage-signatures.spec.ts:127 ✓ Manage Signatures (Moderator) › should apply date filter mod 6.5s mod/manage-signatures.spec.ts:159						

Рисунок 2.19 — Результаты E2E тестирования Playwright

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана комплексная система для верификации цифровых подписей с использованием машинного обучения. Система представляет собой клиент-серверное приложение, построенное на современных технологиях и сетевых протоколах.

Основные результаты работы:

1. Разработано клиент-серверное приложение — создана распределенная система, состоящая из frontend приложения на Next.js, backend inference сервера на FastAPI и базы данных Supabase. Компоненты взаимодействуют через REST API, используя протоколы HTTP/HTTPS для обмена данными.
2. Реализована система верификации подписей — разработан механизм автоматической проверки подлинности подписей с использованием обученной нейронной сети. Система вычисляет векторные представления (эмбединги) подписей и сравнивает их с помощью косинусного сходства, определяя подлинность с точностью, определяемой пороговым значением.
3. Интегрирована ML модель — реализована система управления ML моделями с поддержкой hotswap (переключения моделей без остановки сервера). Модели, обученные в Google Colab, загружаются администратором через веб-интерфейс, автоматически сохраняются в Vercel Blob Storage и синхронизируются с inference сервером для обеспечения высокой доступности.
4. Реализована система безопасности — применены современные методы защиты данных, включая JWT аутентификацию, Row Level Security в базе данных, валидацию входных данных на всех уровнях системы. Обеспечена защита от неавторизованного доступа и SQL инъекций.
5. Создан пользовательский интерфейс — разработан интуитивно понятный веб-интерфейс для работы с подписями, включающий Canvas компонент для рисования, списки подписей, систему верификации и административные панели для управления системой.

6. Проведено модульное и E2E тестирование, проверяющее работу написанно кода отдельных модулей с помощью Jest и связанную работу между frontend, backend и базой данных с помощью Playwright.

Практическая значимость:

Разработанная система может быть применена в различных областях:

- Банковская сфера — для верификации подписей на документах и договорах.
- Юридические документы — для проверки подлинности подписей на контрактах и соглашениях.
- Электронный документооборот — для автоматической проверки подписей в системах управления документами.
- Образование — для проверки подписей на заявлениях и справках.
- Государственные услуги — для верификации подписей в системах электронного правительства

Система может быть расширена в следующих направлениях, для улучшения пользовательского опыта:

- Добавление аспекта игры — создание системы рейтинга между пользователями при создании подделок.
- Интеграция с внешними сервисами — создания системы интерфейсов, позволяющих использовать сторонние нейронные сети для анализа подписей, а также предоставлять API для использования нейронной сети данного сервиса.
- Возможность одновременной работы нескольких нейронных сетей — данный механизм позволит получать разностороннюю оценку поддельных подписей, а также упростить анализ эффективности различных моделей.
- Возможность предоставления доступа к датасету сервиса по запросу — введение системы ключей и журналов для временного предоставления данных подписей для тех, кто не является модератором или администратором.

В результате выполнения курсового проекта были закреплены навыки разработки сетевых приложений, работы с REST API, интеграции различных компонентов системы и применения машинного обучения в практических задачах.

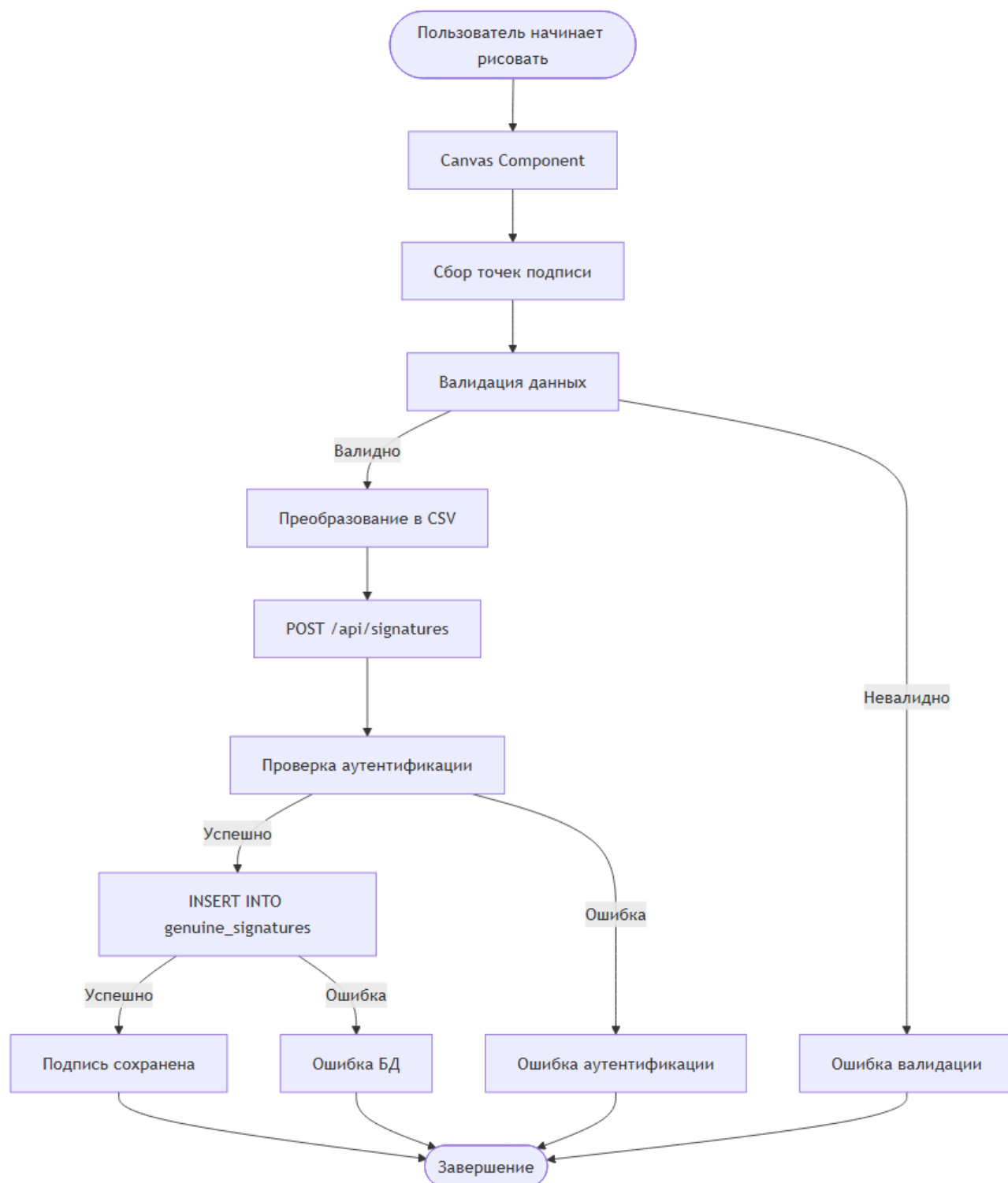
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Context7 – Up-to-date documentation [сайт]. — URL: <https://context7.com> (дата обращения: 13.12.2025).
2. Next.js Docs [сайт]. — URL: <https://nextjs.org/docs> (дата обращения: 13.12.2025).
3. Reference – FastAPI [сайт]. — URL: <https://fastapi.tiangolo.com/ru/reference> (дата обращения: 13.12.2025).
4. Supabase Docs [сайт]. — URL: <https://supabase.com/docs> (дата обращения: 13.12.2025).
5. PostgreSQL: Documentation [сайт]. — URL: <https://www.postgresql.org/docs> (дата обращения: 13.12.2025).
6. RFC 7519 – JSON Web Token (JWT) [сайт]. — URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 13.12.2025).
7. Pydantic Validation [сайт]. — URL: <https://docs.pydantic.dev> (дата обращения: 13.12.2025).
8. 10 Design Patterns in TypeScript [сайт]. — URL: <https://fireship.io/lessons/typescript-design-patterns/> (дата обращения: 13.12.2025).

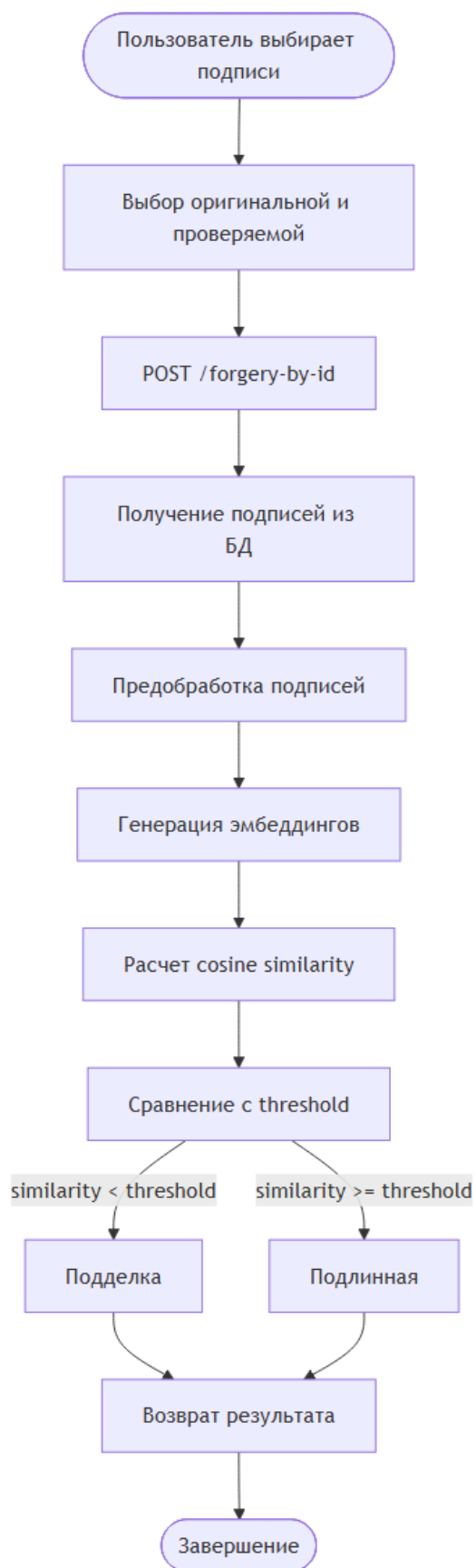
ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Диаграммы потоков данных

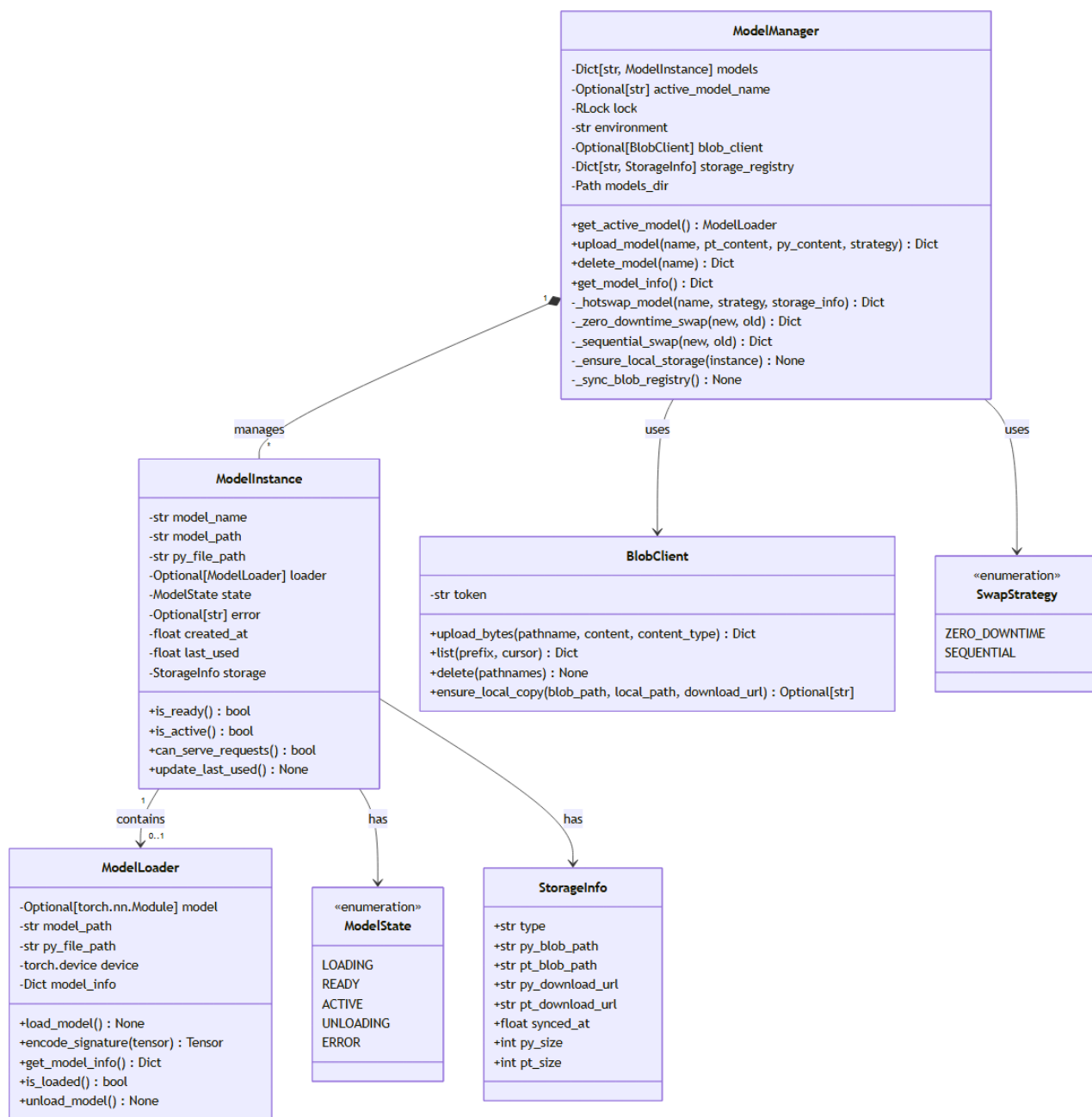


DataFlow создания подписи

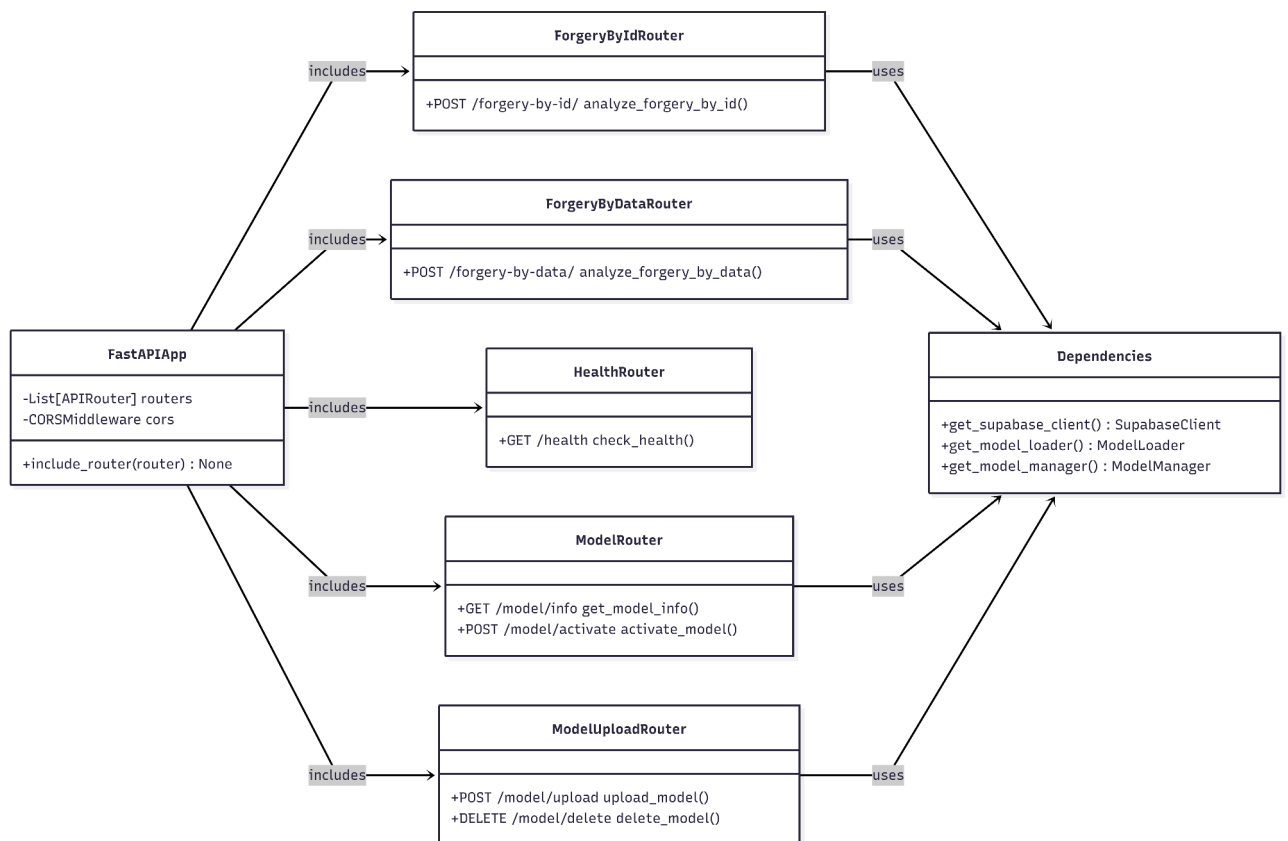


DataFlow верификации подписи

Диаграммы классов

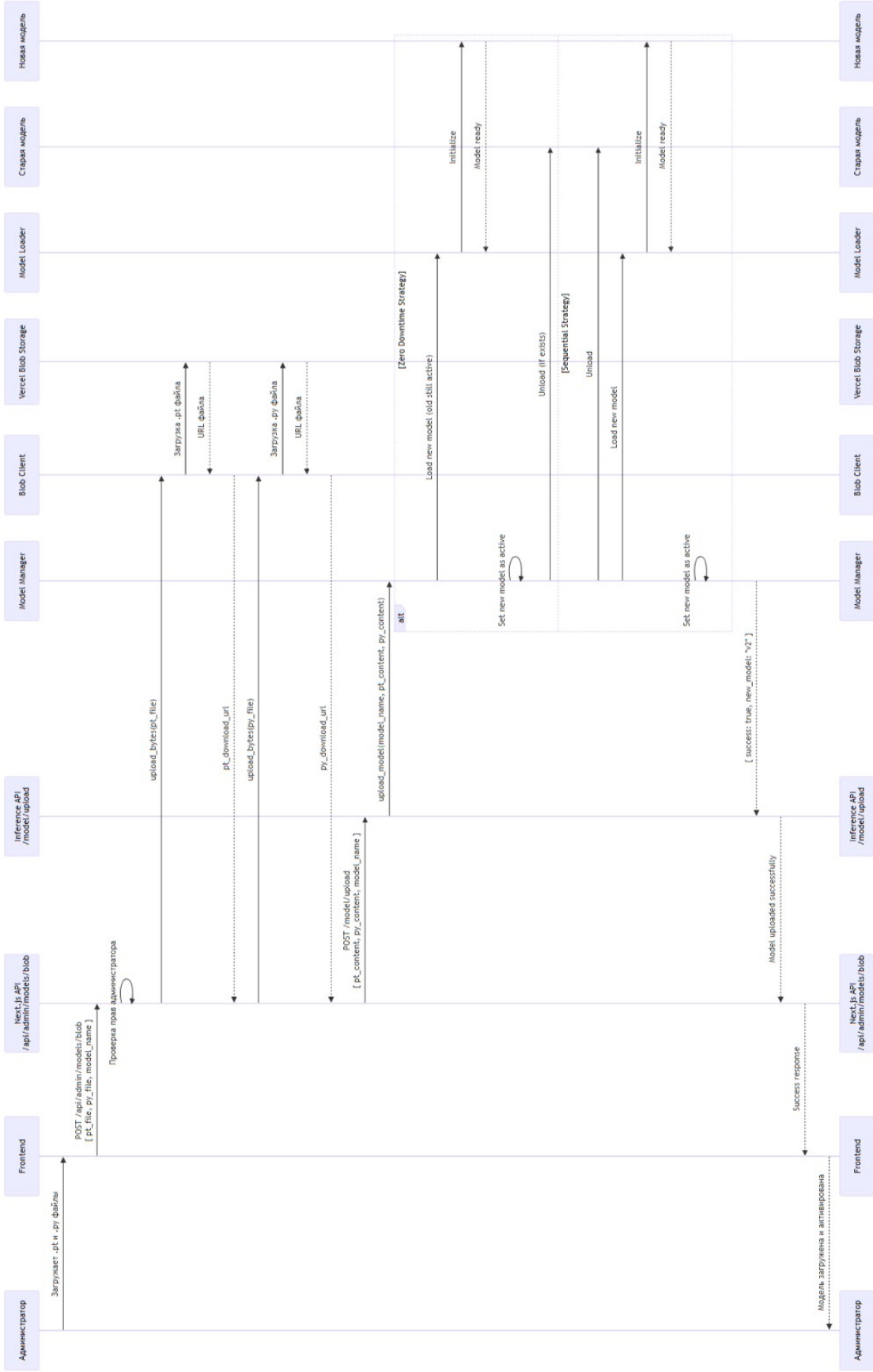


Backend – управление моделями

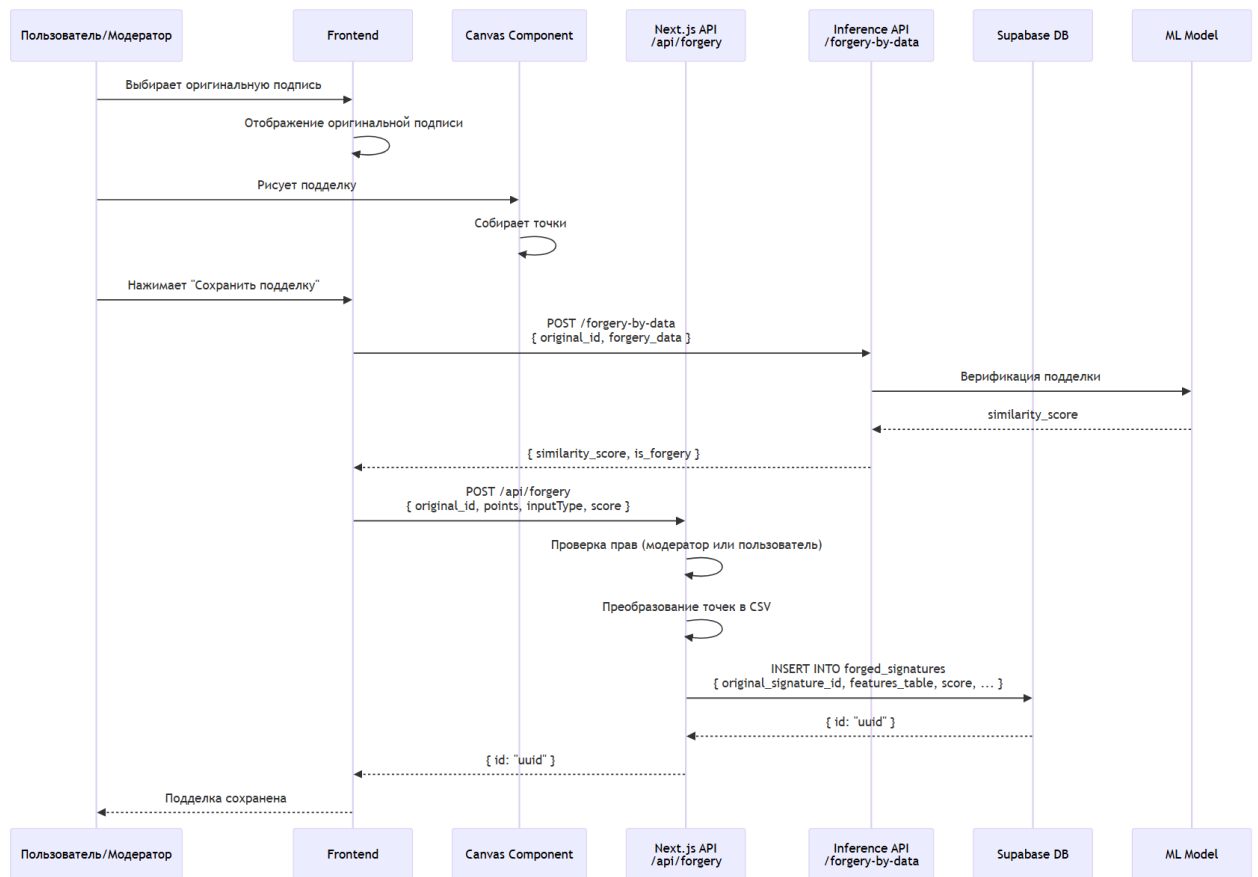


Backend – взаимодействие с Frontend

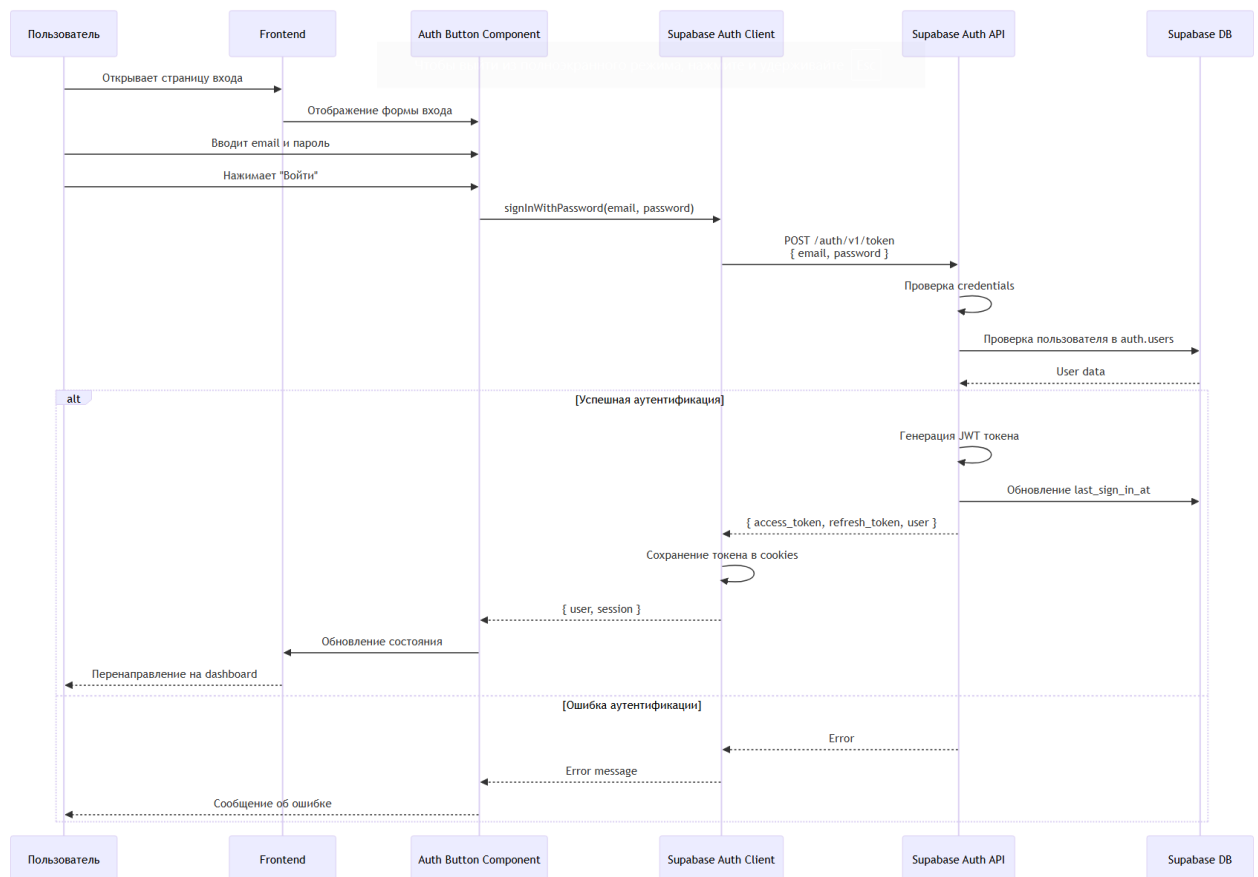
Диаграммы последовательностей



Последовательность загрузки модели нейросети



Создание подделки



Аутентификация пользователя

Диаграмма развертываний и сетевая диаграмма

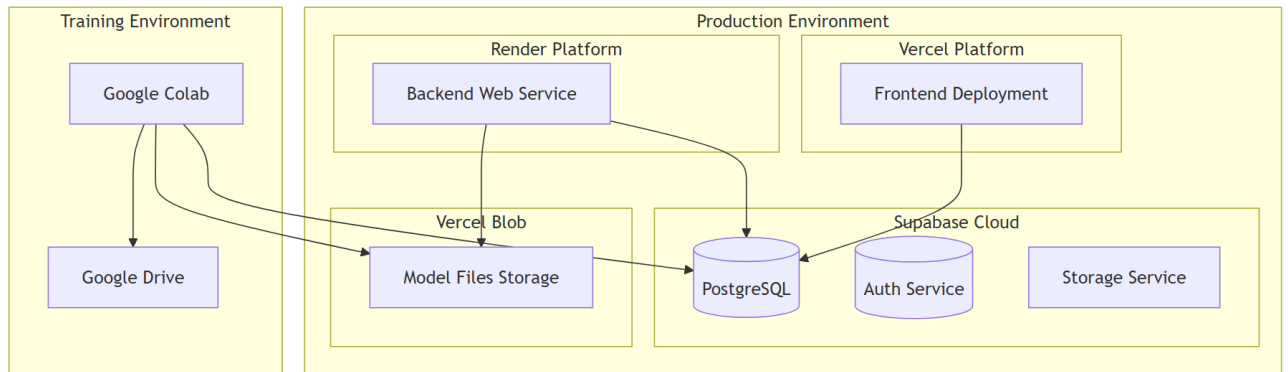
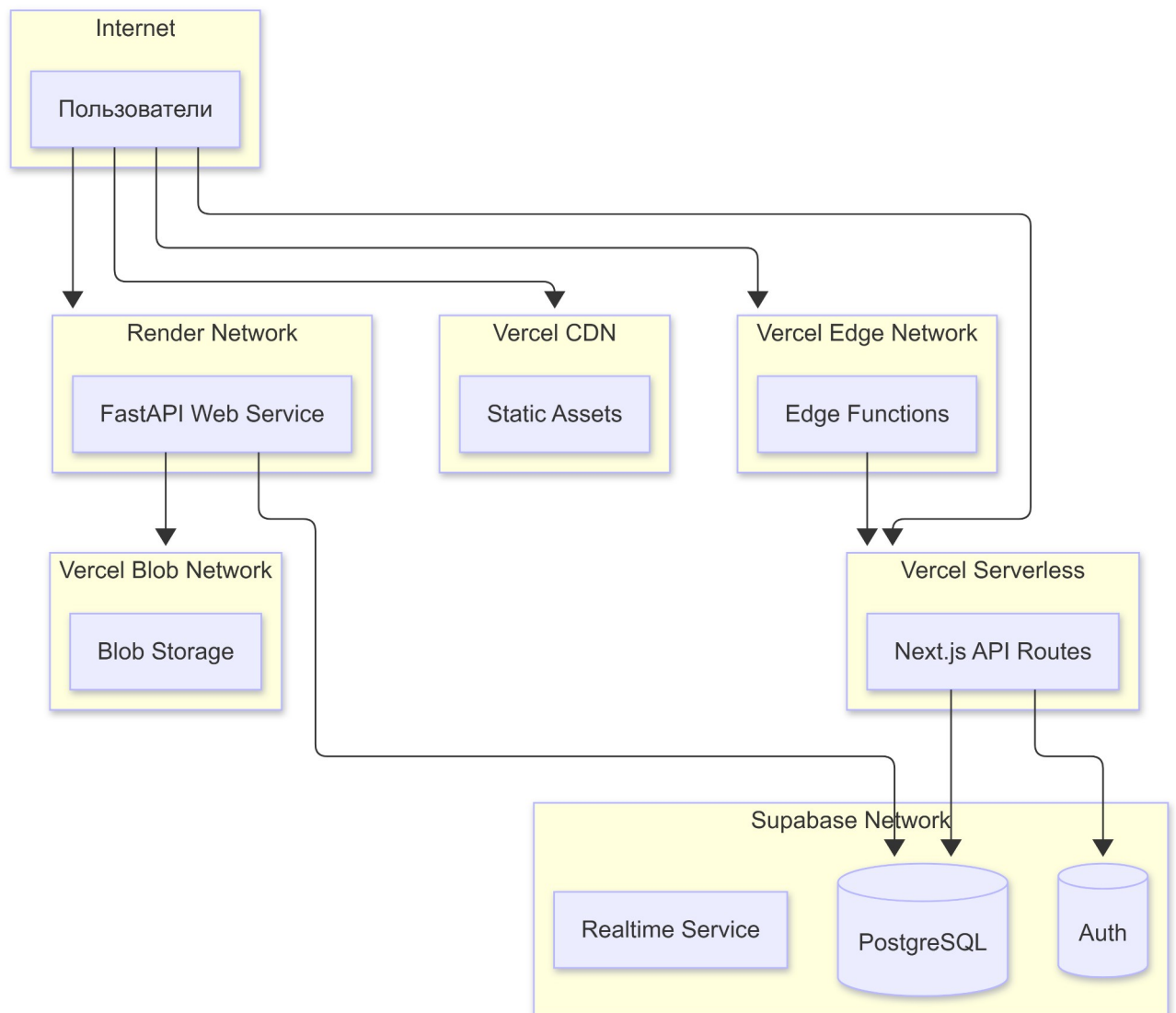


Диаграмма развертываний



Сетевая диаграмма

Код приложения

schema.sql (Код базы данных)

```

CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
CREATE EXTENSION IF NOT EXISTS "vector";

-- Таблица дополнительной информации о пользователе
-- 1:1 с auth.users
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  role VARCHAR(20) NOT NULL DEFAULT 'user' CHECK (role IN ('user', 'mod',
'admin'))),
  display_name VARCHAR(64) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Таблица псевдопользователей для внешних подписей
CREATE TABLE pseudousers (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(64) NOT NULL,
  source VARCHAR(64) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Таблица настоящих подписей
CREATE TABLE genuine_signatures (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  pseudouser_id UUID REFERENCES pseudousers(id) ON DELETE CASCADE,
  features_table TEXT NOT NULL,
  input_type VARCHAR(20) CHECK (input_type IN ('mouse', 'touch', 'pen')),
  user_for_forgery BOOLEAN DEFAULT false,
  mod_for_forgery BOOLEAN DEFAULT true,
  mod_for_dataset BOOLEAN DEFAULT true,
  name VARCHAR(128), -- название файла или что-нибудь еще
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  CONSTRAINT check_exactly_one_user_id CHECK (
    (user_id IS NOT NULL AND pseudouser_id IS NULL) OR
    (user_id IS NULL AND pseudouser_id IS NOT NULL)
  )
);

-- Таблица поддельных подписей
CREATE TABLE forged_signatures (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  original_signature_id UUID REFERENCES genuine_signatures(id) ON DELETE SET
NULL,
  original_user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  original_pseudouser_id UUID REFERENCES pseudousers(id) ON DELETE CASCADE,
  features_table TEXT NOT NULL,
  input_type VARCHAR(20) CHECK (input_type IN ('mouse', 'touch', 'pen'))
  mod_for_dataset BOOLEAN DEFAULT true,
  score NUMERIC(7, 4),
  model_id UUID REFERENCES models(id) ON DELETE CASCADE,
  forger_id UUID REFERENCES profiles(id) ON DELETE SET NULL,
  name VARCHAR(128),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

```

```

        updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
        CONSTRAINT check_exactly_one_user_id CHECK (
            (original_user_id IS NOT NULL AND original_pseudouser_id IS NULL) OR
            (original_user_id IS NULL AND original_pseudouser_id IS NOT NULL)
        )
    );

-- RPC функция для получения случайной подписи для подделки
CREATE OR REPLACE FUNCTION get_random_forgergy_signature()
RETURNS TABLE(id UUID, features_table TEXT)
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
BEGIN
    RETURN QUERY
    SELECT gs.id, gs.features_table
    FROM genuine_signatures gs
    WHERE gs.user_for_forgergy = true
        AND gs.mod_for_forgergy = true
    ORDER BY RANDOM()
    LIMIT 1;
END;
$$;

-- Функция для проверки роли модератора
CREATE OR REPLACE FUNCTION is_mod()
RETURNS BOOLEAN AS $$
BEGIN
    RETURN (
        COALESCE(
            (auth.jwt() ->> 'user_metadata')::jsonb ->> 'role',
            (auth.jwt() ->> 'raw_user_meta_data')::jsonb ->> 'role'
        ) IN ('mod', 'admin')
    );
END;
$$ LANGUAGE plpgsql STABLE SECURITY DEFINER;

-- Функция для проверки роли администратора
CREATE OR REPLACE FUNCTION is_admin()
RETURNS BOOLEAN AS $$
BEGIN
    -- Проверяем роль из JWT claims
    RETURN (
        COALESCE(
            (auth.jwt() ->> 'user_metadata')::jsonb ->> 'role',
            (auth.jwt() ->> 'raw_user_meta_data')::jsonb ->> 'role'
        ) = 'admin'
    );
END;
$$ LANGUAGE plpgsql STABLE SECURITY DEFINER;

-- RLS для таблицы profiles
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;

-- Политика для пользователей: видимость только своего аккаунта
CREATE POLICY "users_can_view_own_profile" ON profiles
FOR SELECT USING (auth.uid() = id);

-- Политика для модераторов: только просмотр всех профилей
CREATE POLICY "mods_can_view_all_profiles" ON profiles
FOR SELECT USING (is_mod());

```

```

-- Политика для админов и service_role: полный доступ к профилям
CREATE POLICY "admins_and_service_full_access_profiles" ON profiles
  FOR ALL USING (
    is_admin() OR auth.role() = 'service_role'
  );

-- RLS для таблицы genuine_signatures
ALTER TABLE genuine_signatures ENABLE ROW LEVEL SECURITY;

-- Политика для пользователей: только чтение своих подписей
CREATE POLICY "users_can_view_own_signatures" ON genuine_signatures
  FOR SELECT USING (auth.uid() = user_id);

-- Политика для админов: полный доступ, исключая подписи других админов
CREATE POLICY "admins_access_genuine_signatures_except_other_admins" ON
genuine_signatures
  FOR ALL USING (
    is_admin() AND (
      user_id IS NULL OR -- псевдопользователи доступны всем админам
      NOT EXISTS (
        SELECT 1 FROM profiles
        WHERE id = user_id
        AND role = 'admin'
        AND id != auth.uid()
      )
    )
  );

-- Политика для модераторов: полный доступ, исключая подписи других модераторов
и админов
CREATE POLICY "mods_access_genuine_signatures_except_other_mods_admins" ON
genuine_signatures
  FOR ALL USING (
    is_mod() AND NOT is_admin() AND (
      user_id IS NULL OR -- псевдопользователи доступны всем модераторам
      NOT EXISTS (
        SELECT 1 FROM profiles
        WHERE id = user_id
        AND role IN ('mod', 'admin')
        AND id != auth.uid()
      )
    )
  );

-- RLS для таблицы forged_signatures
ALTER TABLE forged_signatures ENABLE ROW LEVEL SECURITY;

-- Политика для админов: полный доступ, исключая подписи других админов
CREATE POLICY "admins_access_forged_signatures_except_other_admins" ON
forged_signatures
  FOR ALL USING (
    is_admin() AND (
      original_user_id IS NULL OR -- псевдопользователи доступны всем
      NOT EXISTS (
        SELECT 1 FROM profiles
        WHERE id = original_user_id
        AND role = 'admin'
        AND id != auth.uid()
      )
    )
  );

```

```

-- Политика для модераторов: полный доступ, исключая подписи других модераторов
и админов
CREATE POLICY "mods_access_forged_signatures_except_other_mods_admins" ON
forged_signatures
    FOR ALL USING (
        is_mod() AND NOT is_admin() AND (
            original_user_id IS NULL OR -- псевдопользователи доступны всем
модераторам
            NOT EXISTS (
                SELECT 1 FROM profiles
                WHERE id = original_user_id
                AND role IN ('mod', 'admin')
                AND id != auth.uid()
            )
        )
    );

-- RLS для таблицы pseudousers
ALTER TABLE pseudousers ENABLE ROW LEVEL SECURITY;

-- Политика для модераторов и админов: полный доступ к псевдопользователям
CREATE POLICY "mods_full_access_pseudousers" ON pseudousers
    FOR ALL USING (is_mod());

-- Функция для обновления пользовательских метаданных в JWT
CREATE OR REPLACE FUNCTION set_user_role_in_metadata()
RETURNS TRIGGER AS $$
BEGIN
    -- Обновляем поле raw_user_meta_data в таблице auth.users
    UPDATE auth.users
    SET raw_user_meta_data = jsonb_set(
        coalesce(raw_user_meta_data, '{} '::jsonb),
        '{role}',
        to_jsonb(NEW.role)
    )
    WHERE id = NEW.id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

forgery-by-data.py (Backend inference api route)

```

"""
Роутер для анализа подделки по ID оригинальной подписи и данным поддельной
подписи
"""

```

```

from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from typing import Optional, List, Union
import logging
import torch
import torch.nn.functional as F

from dependencies import get_supabase_client, get_model_loader
from utils.supabase_client import SupabaseClient
from utils.model_loader import ModelLoader
from utils.preprocessing import preprocess_signature_data,
parse_csv_signature_data

logger = logging.getLogger(__name__)

```

```

router = APIRouter(prefix="/forgery-by-data", tags=["forgery-analysis"])

class ForgeryByDataRequest(BaseModel):
    """Схема запроса для анализа подделки по данным."""

    original_id: str
    forgery_data: Union[
        List[List[float]], str
    ] # CSV строка или список списков [t,x,y,p]

class ForgeryAnalysisResponse(BaseModel):
    """Ответ с результатом анализа подделки"""

    is_forgery: bool
    similarity_score: float
    threshold: float
    error: Optional[str] = None

@router.post("/", response_model=ForgeryAnalysisResponse)
async def analyze_forgery_by_data(
    request_body: ForgeryByDataRequest,
    supabase_client: SupabaseClient = Depends(get_supabase_client),
    model_loader: ModelLoader = Depends(get_model_loader),
):
    """
    Анализ подделки по ID оригинальной подписи и данным поддельной подписи

    Args:
        request_body: Валидированное тело запроса
        supabase_client: Клиент Supabase
        model_loader: Загрузчик модели

    Returns:
        Результат анализа подделки
    """
    # Теперь мы получаем ID напрямую из валидированного объекта
    original_id = request_body.original_id

    try:
        logger.info("=== FORGERY BY DATA REQUEST START ===")
        logger.info(f"Analyzing forgery by data: original={original_id}")
        logger.info(f"Forgery data type: {type(request_body.forgery_data)}")

        # --- Шаг 1: Получение данных оригинальной подписи ---
        original_data = supabase_client.get_signature_data(original_id,
            "genuine")
        if not original_data:
            raise HTTPException(
                status_code=404,
                detail=f"Original signature {original_id} not found in genuine
signatures",
            )

        # --- Шаг 2: Обработка данных поддельной подписи ---
        forgery_data: List[List[float]]

        if isinstance(request_body.forgery_data, str):
            # Если это CSV строка, парсим её
            logger.info("Parsing CSV forgery data")
            forgery_data = parse_csv_signature_data(request_body.forgery_data)

```

```

else:
    # Если это уже список списков, используем как есть
    logger.info("Using forgery data as list of lists")
    forgery_data = request_body.forgery_data

    # Проверка валидности данных (для numpy array используем len, для списка
-    обычную проверку)
    if forgery_data is None or len(forgery_data) == 0:
        raise HTTPException(
            status_code=400,
            detail="Invalid forgery data provided or failed to parse",
        )

    # --- Шаг 3: Препроцессинг и подготовка тензоров ---
    # Определяем версию модели из model_loader
    model_info = model_loader.get_model_info()
    model_version = model_info.get("module", "models.v1").split(".")[1] if
    "." in model_info.get("module", "") else "v1"

    original_features = preprocess_signature_data(original_data,
    model_version=model_version)
    forgery_features = preprocess_signature_data(forgery_data,
    model_version=model_version)

    # Преобразуем в тензоры PyTorch
    original_tensor =
    torch.from_numpy(original_features).float().unsqueeze(0)
    forgery_tensor = torch.from_numpy(forgery_features).float().unsqueeze(0)

    # --- Шаг 4: Получение эмбедингов и анализ ---
    original_embedding = model_loader.encode_signature(original_tensor)
    forgery_embedding = model_loader.encode_signature(forgery_tensor)

    # Вычисляем косинусное сходство
    similarity_score = float(
        F.cosine_similarity(original_embedding, forgery_embedding, dim=1)
    )

    # Определяем порог для подделки
    threshold = model_loader.get_model_threshold() or 0.75;

    # Определяем, является ли это подделкой
    is_forgery = similarity_score < threshold

    logger.info(
        f"Analysis completed: similarity={similarity_score:.4f},
is_forgery={is_forgery}"
    )

    result = ForgeryAnalysisResponse(
        is_forgery=is_forgery,
        similarity_score=similarity_score,
        threshold=threshold,
    )

    logger.info(f"=== FORGERY BY DATA REQUEST SUCCESS ===")
    return result

except HTTPException as e:
    logger.error(f"=== FORGERY BY DATA HTTP ERROR ===")
    logger.error(f"HTTP Exception: {e.status_code} - {e.detail}")
    raise
except Exception as e:

```

```
logger.error(f"=== FORGERY BY DATA GENERAL ERROR ===")
logger.error(f"Error analyzing forgery by data: {str(e)}")
import traceback

logger.error(f"Traceback: {traceback.format_exc()}")

# Возвращаем структурированный ответ об ошибке
return ForgeryAnalysisResponse(
    is_forgery=False,
    similarity_score=0.0,
    threshold=threshold,
    error=f"Analysis failed: {type(e).__name__}: {str(e)}",
)
```

Код тестирования

```
// Утилиты для работы с подписями (Jest)
import {
  csvStringToPoints,
  csvToPoints,
  pointsToCSV,
  isValidSignature,
  getSignatureStats,
  prepareGenuineSignatureDataForInsert,
} from '@lib/utils/signature-utils';
import {
  createTestCSV,
  createTestPoints,
  createTestSignature,
} from '@lib/__tests__/test-helpers';
import { SignaturePoint } from '@lib/types';

describe('signature-utils', () => {
  describe('pointsToCSV', () => {
    it('should convert points array to CSV string', () => {
      const points: SignaturePoint[] = [
        { timestamp: 1000, x: 10, y: 20, pressure: 0.5 },
        { timestamp: 1010, x: 15, y: 25, pressure: 0.6 },
      ];

      const csv = pointsToCSV(points);

      expect(csv).toBe('t,x,y,p\n1000,10,20,0.5\n1010,15,25,0.6');
    });

    it('should handle empty array', () => {
      const csv = pointsToCSV([]);
      expect(csv).toBe('t,x,y,p\n');
    });
  });

  describe('csvStringToPoints', () => {
    it('should convert CSV string to points array', () => {
      const csv = 't,x,y,p\n1000,10,20,0.5\n1010,15,25,0.6';
      const points = csvStringToPoints(csv);

      expect(points).toEqual([
        { timestamp: 1000, x: 10, y: 20, pressure: 0.5 },
        { timestamp: 1010, x: 15, y: 25, pressure: 0.6 },
      ]);
    });

    it('should handle CSV with whitespace', () => {
      const csv = 't,x,y,p\n 1000,10,20,0.5 \n 1010,15,25,0.6 ';
      const points = csvStringToPoints(csv);

      expect(points).toEqual([
        { timestamp: 1000, x: 10, y: 20, pressure: 0.5 },
        { timestamp: 1010, x: 15, y: 25, pressure: 0.6 },
      ]);
    });
  });

  describe('round-trip conversion', () => {

```



```

    it('should convert points to CSV and back correctly', () => {
      const originalPoints = createTestPoints(10);
      const csv = pointsToCSV(originalPoints);
      const convertedPoints = csvStringToPoints(csv);

      expect(convertedPoints).toEqual(originalPoints);
    });
  });

describe('isValidSignature', () => {
  it('should return true for signature with enough points', () => {
    const signature = createTestSignature('genuine', {
      points: createTestPoints(15),
    });

    expect(isValidSignature(signature, 10)).toBe(true);
  });

  it('should return false for signature with too few points', () => {
    const signature = createTestSignature('genuine', {
      points: createTestPoints(5),
    });

    expect(isValidSignature(signature, 10)).toBe(false);
  });
});

describe('getSignatureStats', () => {
  it('should calculate stats for signature with points', () => {
    const points: SignaturePoint[] = [
      { timestamp: 1000, x: 10, y: 20, pressure: 0.5 },
      { timestamp: 1010, x: 15, y: 25, pressure: 0.6 },
      { timestamp: 1020, x: 20, y: 30, pressure: 0.7 },
    ];
    const signature = createTestSignature('genuine', { points });

    const stats = getSignatureStats(signature);

    expect(stats.pointCount).toBe(3);
    expect(stats.duration).toBe(0.02); // (1020 - 1000) / 1000
    expect(stats.averagePressure).toBeCloseTo(0.6, 1);
    expect(stats.bounds.minX).toBe(10);
    expect(stats.bounds.maxX).toBe(20);
    expect(stats.bounds.minY).toBe(20);
    expect(stats.bounds.maxY).toBe(30);
  });
});

describe('prepareGenuineSignatureDataForInsert', () => {
  it('should prepare data with user_id', () => {
    const points = createTestPoints(10);
    const data = prepareGenuineSignatureDataForInsert(
      points,
      'mouse',
      'user-123',
      null,
      false
    );

    expect(data.user_id).toBe('user-123');
    expect(data.pseudouser_id).toBeUndefined();
    expect(data.features_table).toBe(createTestCSV(points));
    expect(data.input_type).toBe('mouse');
  });
});

```

```

    });

    it('should throw error when both user_id and pseudouser_id are provided', ()
=> {
    const points = createTestPoints(10);

    expect(() => {
      prepareGenuineSignatureDataForInsert(
        points,
        'mouse',
        'user-123',
        'pseudo-123'
      );
    }).toThrow('Only one of user or pseudouser id is allowed');
  });
});
});
});

```

// E2E верификация (Playwright)

```

import { test, expect } from '@playwright/test';

test.describe('Analyze Forgery (User)', () => {
  test.beforeEach(async ({ page }) => {
    // Переходим на главную страницу (где происходит анализ подделки)
    await page.goto('/', { waitUntil: 'networkidle' });
    await page.waitForLoadState('domcontentloaded');
    await page.waitForTimeout(2000); // Даем время на загрузку подписи с сервера
  });

  test('should analyze forgery by drawing on canvas', async ({ page }) => {
    test.setTimeout(60000); // Увеличиваем таймаут для этого теста
    const canvas = page.locator('canvas').first();
    await expect(canvas).toBeVisible({ timeout: 10000 });

    // Получаем размеры canvas
    const box = await canvas.boundingBox();
    if (!box) {
      throw new Error('Canvas not found');
    }

    // Рисуем подделку (минимум 20 точек)
    const centerX = box.x + box.width / 2;
    const centerY = box.y + box.height / 2;

    await page.mouse.move(centerX - 50, centerY);
    await page.mouse.down();

    // Рисуем подпись (создаем минимум 25 точек)
    for (let i = 0; i < 25; i++) {
      const x = centerX - 50 + i * 4;
      const y = centerY + Math.sin(i * 0.3) * 15;
      await page.mouse.move(x, y, { steps: 1 });
      await page.waitForTimeout(10);
    }

    await page.mouse.up();
    await page.waitForTimeout(500);

    // Нажимаем кнопку анализа
    const analyzeButton = page
      .getByRole('button', {
        name: /Анализировать подпись/i,

```

```

    })
    .first();

await analyzeButton.click();
await page.waitForTimeout(200);

// Ждем завершения анализа
await page.waitForTimeout(3000);

// Проверяем что анализ завершился - либо появилось модальное окно
try {
    const modalTitle = page.getByText(/Результат анализа подделки/i).first();
    await expect(modalTitle).toBeVisible({ timeout: 25000 });

    // Если модальное окно найдено, проверяем наличие результата
    const resultText = page
        .getByText(/Схожесть|Подлинная|Поддельная/i)
        .first();
    await expect(resultText).toBeVisible({ timeout: 5000 });
} catch {
    // Если модальное окно не появилось, проверяем что анализ завершился
    const analyzeButtonAfterAnalysis = page
        .getByRole('button', {
            name: /Анализировать подпись/i,
        })
        .first();
    await expect(analyzeButtonAfterAnalysis).toBeEnabled({ timeout: 25000 });
}
});

test('should display similarity score in result', async ({ page }) => {
    test.setTimeout(60000);
    const canvas = page.locator('canvas').first();
    const box = await canvas.boundingBox();
    if (!box) {
        throw new Error('Canvas not found');
    }

    // Рисуем подпись
    const centerX = box.x + box.width / 2;
    const centerY = box.y + box.height / 2;

    await page.mouse.move(centerX - 50, centerY);
    await page.mouse.down();

    for (let i = 0; i < 25; i++) {
        const x = centerX - 50 + i * 4;
        const y = centerY + Math.sin(i * 0.3) * 15;
        await page.mouse.move(x, y, { steps: 1 });
        await page.waitForTimeout(10);
    }

    await page.mouse.up();
    await page.waitForTimeout(500);

    // Анализируем
    const analyzeButton = page
        .getByRole('button', {
            name: /Анализировать подпись/i,
        })
        .first();
    await analyzeButton.click();

```

```

await page.waitForTimeout(3000);

// Пытаемся найти модальное окно с результатом
try {
  const modalTitle = page.getByText(/Результат анализа подделки/i).first();
  await expect(modalTitle).toBeVisible({ timeout: 25000 });

  // Проверяем наличие текста "Схожесть:"
  const similarityLabel = page.getByText(/Схожесть:/i).first();
  await expect(similarityLabel).toBeVisible({ timeout: 5000 });

  // Проверяем наличие процента схожести
  const similarityPercent = page.getByText(/\d+%/).first();
  await expect(similarityPercent).toBeVisible({ timeout: 5000 });
} catch {
  // Если модальное окно не появилось, проверяем что анализ завершился
  const analyzeButton = page
    .getByRole('button', {
      name: /Анализировать подпись/i,
    })
    .first();
  await expect(analyzeButton).toBeEnabled({ timeout: 25000 });
}
});
});

```